



MIXED REALITY-KINO MIT FREUNDEN GRUPPE 1

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Technischer Entwurf

Auftraggeber

Technische Universität Braunschweig
Institut für Werkzeugmaschinen und Fertigungstechnik
Prof. Dr.-Ing. Klaus Dröder
Langer Kamp 19b
38106 Braunschweig

Betreuer: Naomi Beyer, Benjamin Effner

Auftragnehmer:

Name	E-Mail-Adresse
Seif Saad	s.saad@tu-braunschweig.de
Dhouha Khalfalli Ep Ghannouchi	d.khalfalli-ep-ghannouchi@tu-braunschweig.de
Yasmine Slimane	y.slimane@tu-braunschweig.de
Fadoua Toumi	f.toumi@tu-braunschweig.de
Amer Zaitounh	a.zaitounh@tu-braunschweig.de

Braunschweig, 9. Juli 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Seif Saad	
2	Yasmine Slimane	
2.1	Yasmine Slimane	
2.2	Yasmine Slimane	
2.3	Yasmine Slimane	
2.4	Yasmine Slimane	
2.5	Yasmine Slimane	
2.6	Amer Zaitounh	
2.7	Amer Zaitounh	
2.8	Amer Zaitounh	
2.9	Amer Zaitounh	
3	Yasmine Slimane	
3.1	Yasmine Slimane	
3.2	Yasmine Slimane	
3.3	Seif Saad	
4	Yasmine Slimane	
5	Amer Zaitounh	
5.1	Amer Zaitounh	
6	Fadoua Toumi	
6.1	Fadoua Toumi	
6.2	Fadoua Toumi	
7	Dhouha Khalfalli	
8	Dhouha Khalfalli	
9	Fadoua Toumi	
9.1	Fadoua Toumi	
9.2	Fadoua Toumi	
9.3	Fadoua Toumi	
10	Dhouha Khalfalli, Yasmine Slimane	

Inhaltsverzeichnis

1	Einleitung	6
1.1	Projektdetails	8
2	Analyse der Produktfunktionen	9
2.1	Analyse von Funktionalität <F10>: <Initialisierung und Start des Mixed-Reality-Kinos>	10
2.2	Analyse von Funktionalität <F20>: <Filmauswahl>	11
2.3	Analyse von Funktionalität <F30>: <Szenenauswahl>	12
2.4	Analyse von Funktionalität <F31>: <Film im Mixed-Reality-Overlay>	13
2.5	Analyse von Funktionalität <F32>: <Film in virtueller Umgebung>	14
2.6	Analyse von Funktionalität <F40>: <Steuerung (Play, Pause, Stopp)>	15
2.7	Analyse von Funktionalität <F50>: <Menüdarstellung im Mixed-Reality-Raum>	16
2.8	Analyse von Funktionalität <F60>: <Multiplayer-Synchronisation (gleichzeitiger Filmstart/-stopp)>	17
2.9	Analyse von Funktionalität <F70>: <Verbindungsaufbau und Management über WLAN>	18
3	Resultierende Softwarearchitektur	19
3.1	Komponentenspezifikation	20
3.2	Schnittstellenspezifikation	22
3.3	Protokolle für die Benutzung der Komponenten	23
3.3.1	Protokoll für <C10>: Host Logic	24
3.3.2	Protokoll für <C20>: Client Logic	25
3.3.3	Protokoll für <C30>: RPC Handler	26
3.3.4	Protokoll für <C40>: User Interface	27
3.3.5	Protokoll für <C50>: Video Player	28
3.3.6	Protokoll für <C60>: Netzwerk Manager	29
3.3.7	Protokoll für <C70>: Video Server (extern)	30
4	Verteilungsentwurf	31

5 Implementierungsentwurf	32
5.1 Implementierung von Komponente <C10>: <SessionManager>	33
5.1.1 Paket-/Klassendiagramm	33
5.1.2 Erläuterung	34
5.2 Implementierung von Komponente <C20>: <MediaController>	35
5.2.1 Paket-/Klassendiagramm	35
5.2.2 Erläuterung	36
5.3 Implementierung von Komponente <C30>: <UIManager>	37
5.3.1 Paket-/Klassendiagramm	37
5.3.2 Erläuterung	38
5.4 Implementierung von Komponente <C40>: <EnvironmentLoader>	39
5.4.1 Paket-/Klassendiagramm	39
5.4.2 Erläuterung	40
5.5 Implementierung von Komponente <C50>: <NetworkManager>	41
5.5.1 Paket-/Klassendiagramm	41
5.5.2 Erläuterung	42
6 Datenmodell	43
6.1 Diagramm	44
6.2 Erläuterung	44
7 Konfiguration	47
8 Änderungen gegenüber Fachentwurf	48
9 Erfüllung der Kriterien	49
9.1 Musskriterien	49
9.2 Sollkriterien	50
9.3 Kannkriterien	51
10 Glossar	52

Abbildungsverzeichnis

1.1	Aktivitätsdiagramm des Hauptnutzungsszenarios	7
2.1	Initialisierung und Kalibrierung	10
2.2	Filmauswahl	11
2.3	Szenenauswahl	12
2.4	Film im MR	13
2.5	Film im VR	14
2.6	Steuerung (Play, Pause, Stopp)	15
2.7	Menüdarstellung im Mixed-Reality-Raum	16
2.8	Multiplayer-Synchronisation (gleichzeitiger Filmstart/-stopp)	17
2.9	Verbindungsaufbau und Management über WLAN	18
3.1	Komponentendiagramm	20
3.2	State-Chart der Host Logic-Komponente	24
3.3	State-Chart der Client Logic-Komponente	25
3.4	State-Chart der RPC-Komponente	26
3.5	State-Chart der UI-Komponente	27
3.6	State-Chart der Video Player-Komponente	28
3.7	State-Chart der Netzwerk Manager-Komponente	29
3.8	State-Chart der Server-Komponente	30
4.1	Verteilungsdiagramm	31
5.1	Klassendiagramm für Komponente $\langle C10 \rangle$: SessionManager	33
5.2	Klassendiagramm für Komponente $\langle C20 \rangle$: MediaController	35
5.3	Klassendiagramm für Komponente $\langle C30 \rangle$: UIManager	37
5.4	Klassendiagramm für Komponente $\langle C40 \rangle$: EnvironmentLoader	39
5.5	Klassendiagramm für Komponente $\langle C50 \rangle$: NetworkManager	41
6.1	Klassendiagramm	44

1 Einleitung

In diesem Dokument sollen dem Auftraggeber die einzelnen Funktionen unseres Produkts nähergebracht und detaillierter beschrieben werden. Dafür wird im ersten Abschnitt zunächst ein Aktivitätsdiagramm gezeigt, auf das in den folgenden Teilabschnitten näher eingegangen wird, damit die Vorgehensweise aufgeklärt wird. Im zweiten Abschnitt werden dann die im Pflichtenheft Abschnitt 4 bereits kurz vorgestellten Produktfunktionen näher beschrieben und die genauen Interaktionen des Codes miteinander aufgezeigt. Danach wird im dritten Abschnitt die Softwarearchitektur beschrieben, sowie mit einem Komponentendiagramm grafisch dargestellt. Im vierten Abschnitt wird dann kurz ein Verteilungsdiagramm unseres Projekts vorgelegt, anschließend werden die einzelnen Klassen und Pakete im fünften Abschnitt detaillierter ausgeführt. Die gespeicherten Daten, die im Pflichtenheft Abschnitt 5 kurz dargestellt wurden, werden hier im sechsten Abschnitt näher ausgearbeitet, bevor im siebten Abschnitt des Dokuments näher auf die Rechnerkonfiguration sowie die Parameter des Programms eingegangen wird. Im achten Abschnitt wird kurz auf die umgesetzten Änderungen zwischen dem Fachentwurf und diesem Dokument eingegangen. Als letztes wird noch definiert, durch welche Komponenten aus Abschnitt 3 dieses Dokuments die einzelnen Kriterien umgesetzt werden, die im Pflichtenheft zuerst aufgestellt wurden.

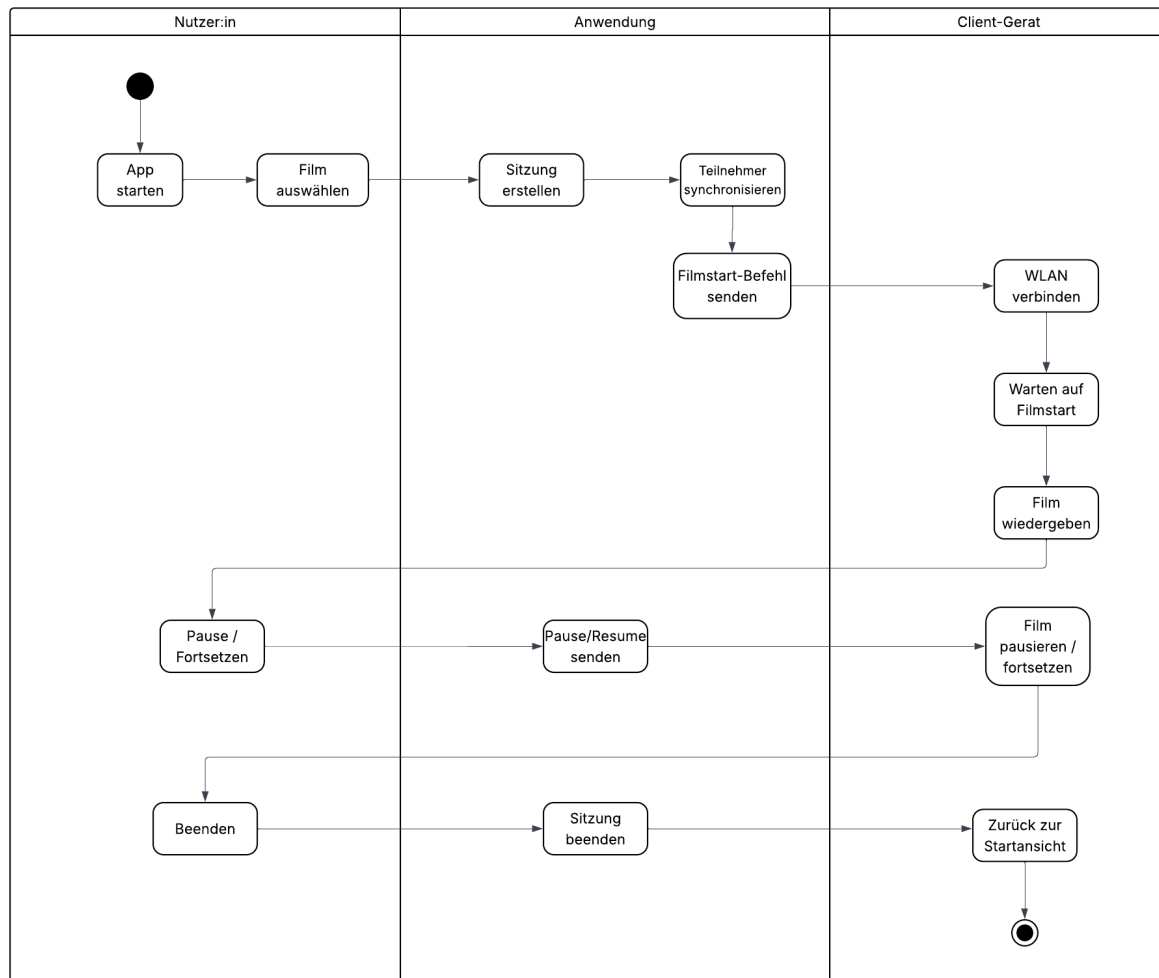


Abbildung 1.1: Aktivitätsdiagramm des Hauptnutzungsszenarios

1.1 Projektdetails

Die Anwendung „Mixed-Reality-Kino mit Freunden“ ermöglicht es mehreren Nutzer:innen, gemeinsam einen Film in einer virtuellen Umgebung zu erleben. Über VR-Headsets (Meta Quest 3) wird eine synchronisierte Wiedergabe gestartet, wobei eine Person die Rolle des Hosts übernimmt. Die übrigen Teilnehmenden verbinden sich über ein lokales WLAN mit der Anwendung und erhalten von dort Steuerbefehle.

Der Ablauf des Systems ist im Aktivitätsdiagramm in Abbildung 1.1 dargestellt. Zunächst wird die App gestartet, und die Nutzer:innen wählen einen Film aus. Die Anwendung erstellt daraufhin eine Sitzung und synchronisiert alle verbundenen Geräte. Nachdem die Kalibrierung und Synchronisation abgeschlossen ist, wird die Wiedergabe gestartet. Der Host kann während der Laufzeit Steuerbefehle wie Pause, Fortsetzen oder Beenden senden, die direkt an alle Geräte verteilt und synchron umgesetzt werden.

Im Falle einer Trennung (z. B. durch WLAN-Verlust) ist ein automatischer Reconnect-Mechanismus vorgesehen. Nach Beenden der Wiedergabe wird die Sitzung geschlossen und alle Geräte kehren in den Ausgangszustand zurück.

2 Analyse der Produktfunktionen

In diesem Kapitel werden die zentralen Funktionalitäten der Anwendung „*Mixed Reality Kino mit Freunden*“ systematisch analysiert und mittels Sequenzdiagrammen visualisiert. Die Analyse basiert auf den im Pflichtenheft definierten Musskriterien und zeigt, wie die Systemkomponenten interagieren, um diese Anforderungen zu erfüllen. Jede Produktfunktion wird in einem eigenen Unterkapitel behandelt und umfasst eine detaillierte Betrachtung mittels Sequenzdiagrammen und ergänzenden Beschreibungen.

2.1 Analyse von Funktionalität <F10>: <Initialisierung und Start des Mixed-Reality-Kinos>

Die Funktion <F10> startet eine gemeinsame Mixed-Reality-Sitzung. Alle Nutzer:innen verbinden sich mit derselben Sitzung. Danach beginnt automatisch die Kalibrierung, um sicherzustellen, dass alle Geräte dieselbe Szene gleich ausgerichtet anzeigen.

Die Kalibrierung nutzt Referenzpunkte oder Marker. Jedes Gerät richtet sich daran aus und sendet die ermittelte Position an das System. Das System prüft alle Daten und synchronisiert die Szene für alle Clients. Erst nach erfolgreicher Kalibrierung ist die Sitzung bereit für weitere Schritte wie Kartenauswahl und Filmstart.

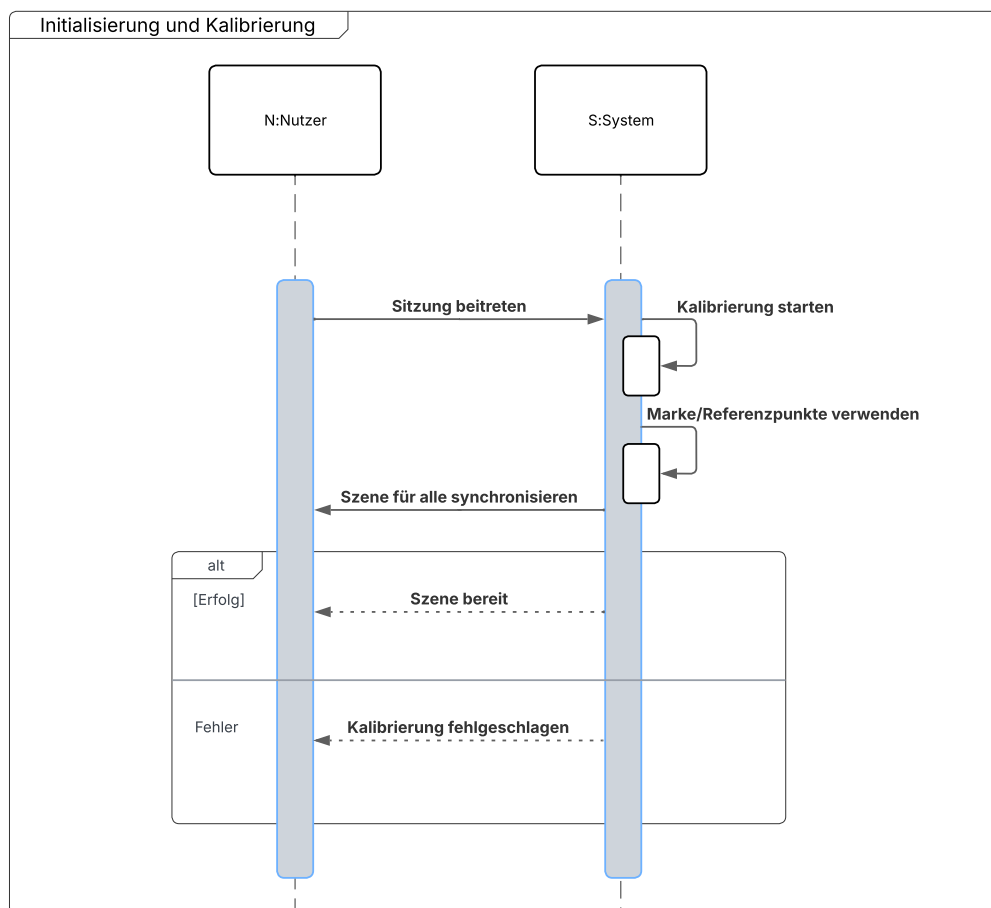


Abbildung 2.1: Initialisierung und Kalibrierung

2.2 Analyse von Funktionalität <F20>: <Filmauswahl>

Diese Funktion <F20> erlaubt allen verbundenen Nutzer:innen, gemeinsam über einen Film abzustimmen. Eine Liste mit verfügbaren Filmen wird angezeigt. Jede Person gibt ihre Stimme ab. Das System zählt die Stimmen aus und zeigt das Ergebnis.

Der Film mit den meisten Stimmen wird anschließend automatisch gestartet. Bei Gleichstand entscheidet das System zufällig oder nach definierten Regeln.

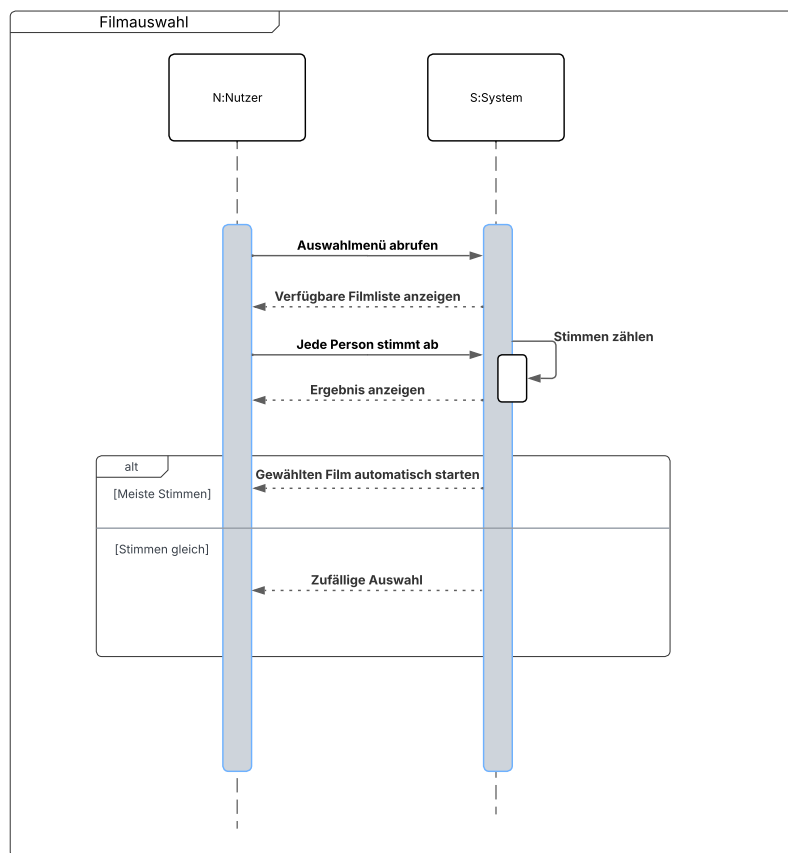


Abbildung 2.2: Filmauswahl

2.3 Analyse von Funktionalität <F30>: <Szenenauswahl>

Diese Funktion <F20> ermöglicht es Nutzer:innen, nach der Filmauswahl den gewünschten Darstellungsmodus zu wählen: zwischen Mixed-Reality oder einer virtuellen Umgebung.

Nach dem Öffnen des Szenenauswahlmenüs wird eine Liste verfügbarer Szenen angezeigt. Eine Szene wird ausgewählt, und das System lädt die entsprechende Umgebung. Bei einem Fehler wird eine Fehlermeldung angezeigt.

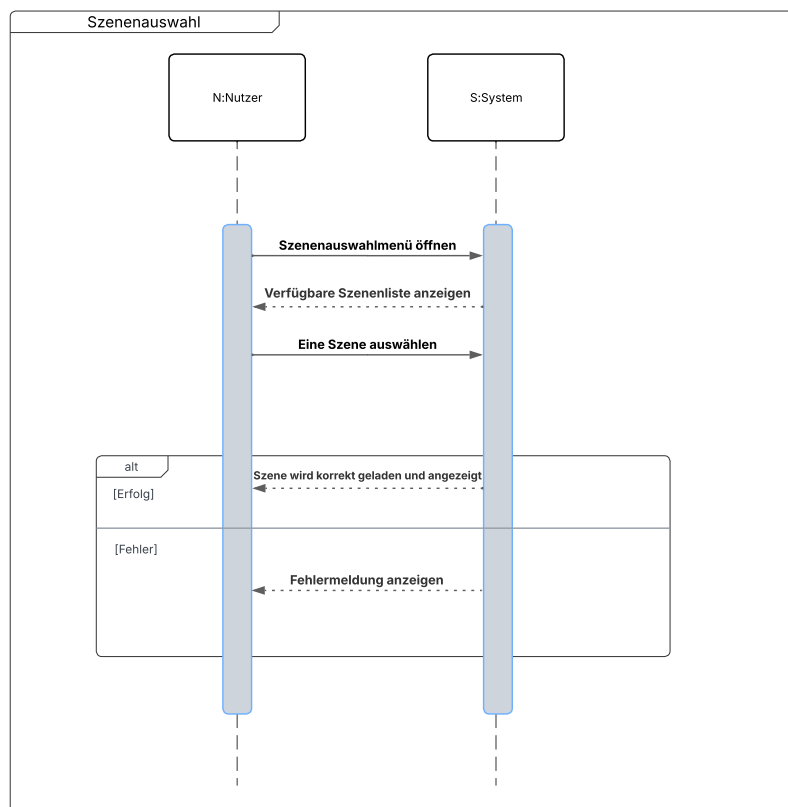


Abbildung 2.3: Szenenauswahl

2.4 Analyse von Funktionalität <F31>: <Film im Mixed-Reality-Overlay>

Diese Funktion ermöglicht das Einblenden einer virtuellen Leinwand in die reale Umgebung. Sie wird aktiviert, sobald der MR-Modus gestartet ist.

Zunächst wird die Kamera des Geräts eingeschaltet. Anschließend wird eine virtuelle Leinwand im realen Raum positioniert und auf dem Videobild eingeblendet. Die Ausrichtung der Leinwand erfolgt anhand festgelegter Referenzpunkte, sodass sie stabil und korrekt im Raum erscheint.

Die Nutzer:innen sehen dadurch eine gemischte Realität und können den Film direkt auf der virtuellen Leinwand anschauen.

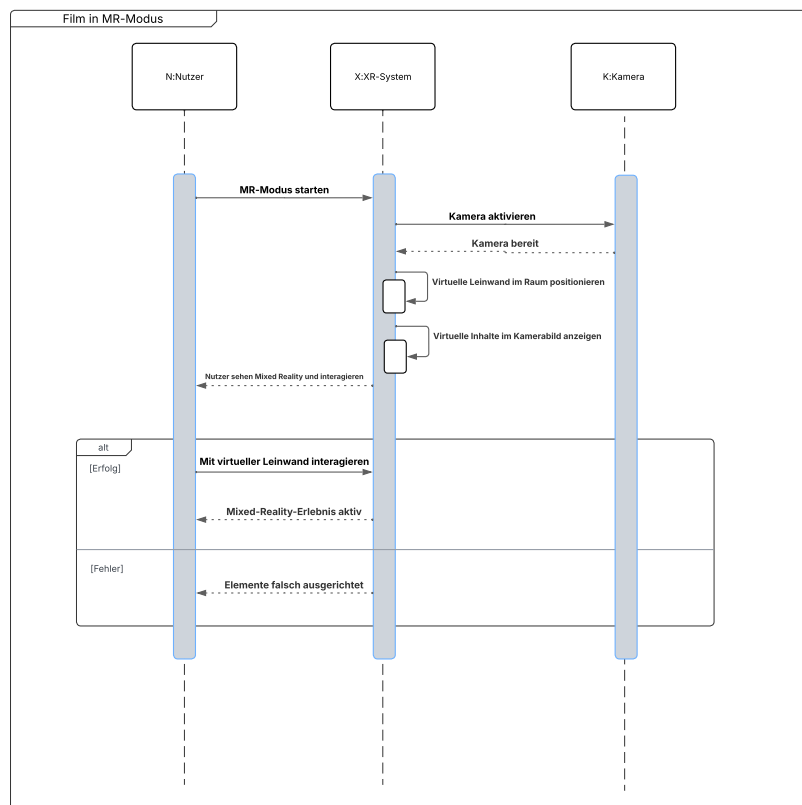


Abbildung 2.4: Film im MR

2.5 Analyse von Funktionalität <F32>: <Film in virtueller Umgebung>

Diese Funktion ermöglicht es, den Film gemeinsam in einer ausgewählten VR-Szene anzusehen (im Wohnzimmer oder Kinosaal).

Nach Auswahl einer Szene im VR-Modus wird diese Umgebung geladen und dargestellt. Die Nutzer:innen können sich darin umsehen und hinsetzen. Sobald alle bereit sind, wird der Film synchron gestartet. Ziel ist ein immersives Filmerlebnis mit realistisch wirkender Umgebung.

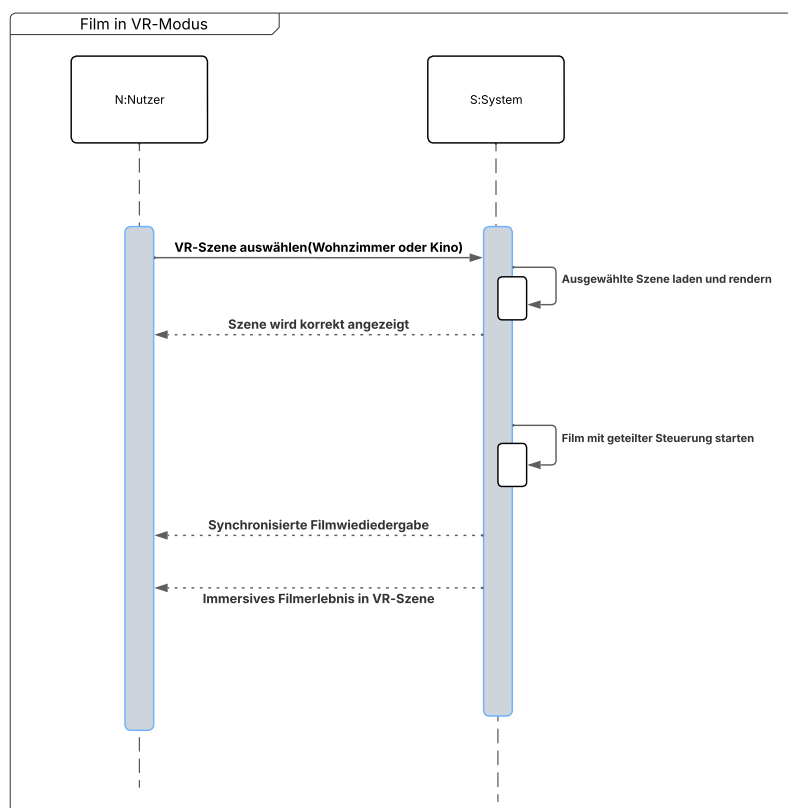


Abbildung 2.5: Film im VR

2.6 Analyse von Funktionalität <F40>: <Steuerung (Play, Pause, Stopp)>

Die Funktion <F40> ermöglicht die zentrale Steuerung der Videowiedergabe innerhalb einer Mixed-Reality-Sitzung. Ein:e Host-Nutzer:in kann die Wiedergabe starten, pausieren oder stoppen. Diese Befehle werden an alle Clients im Netzwerk gleichzeitig übermittelt.

Die Geräte reagieren synchron auf die Eingaben, sodass bei allen Teilnehmer:innen dieselbe Aktion gleichzeitig ausgeführt wird. Das System verhindert dabei doppelte oder widersprüchliche Eingaben, um Konsistenz sicherzustellen.

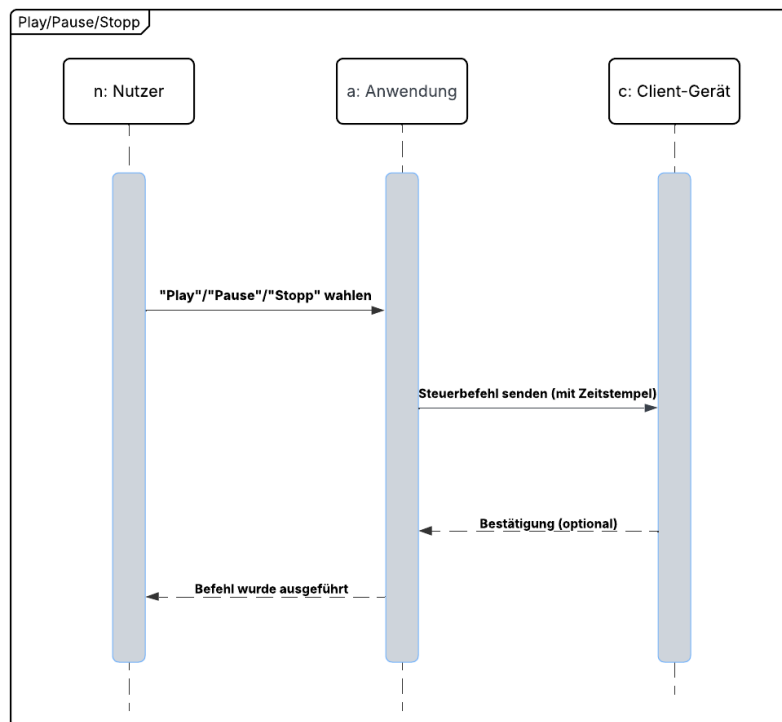


Abbildung 2.6: Steuerung (Play, Pause, Stopp)

2.7 Analyse von Funktionalität <F50>: <Menüdarstellung im Mixed-Reality-Raum>

Die Funktion <F50> sorgt dafür, dass das Menü zur Steuerung und Auswahl in der Mixed-Reality-Umgebung jederzeit sichtbar und lesbar bleibt.

Das Menü wird in einem festen Bereich im Raum angezeigt, der für alle Nutzer:innen gleich ausgerichtet ist. Es passt sich der Raumbeleuchtung an und bleibt auch bei Bewegung oder unterschiedlichen Blickrichtungen stabil.

Die Steuerung erfolgt über Handgesten oder Controller-Eingaben. So wird sichergestellt, dass die Bedienung auch in verschiedenen Lichtverhältnissen oder Entfernungen funktioniert.

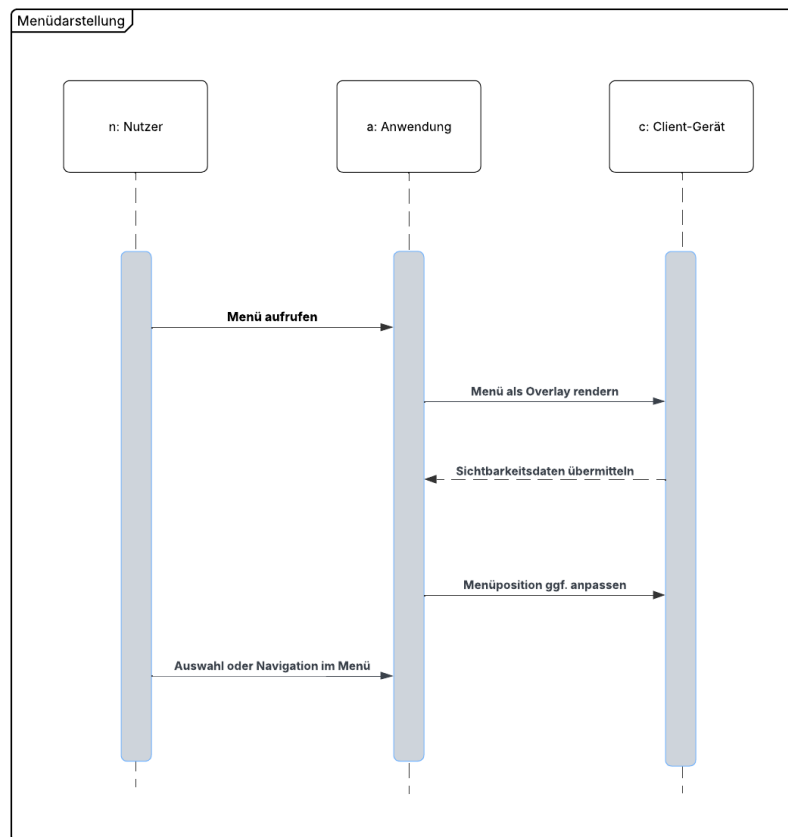


Abbildung 2.7: Menüdarstellung im Mixed-Reality-Raum

2.8 Analyse von Funktionalität <F60>: <Multiplayer-Synchronisation (gleichzeitiger Filmstart/-stopp)>

Die Funktion <F60> stellt sicher, dass der Start und Stopp eines Films bei allen Teilnehmenden exakt gleichzeitig erfolgt.

Ein globaler Befehl mit Zeitstempel wird vom Host an alle Clients gesendet. Die Geräte stimmen sich intern ab, um mit möglichst geringer Abweichung (unter 0,5 Sekunden) den Film synchron zu starten oder zu stoppen.

Bei Verbindungsverlust oder Verzögerung prüft das System automatisch den Zustand und versucht, betroffene Geräte wieder korrekt zu synchronisieren.

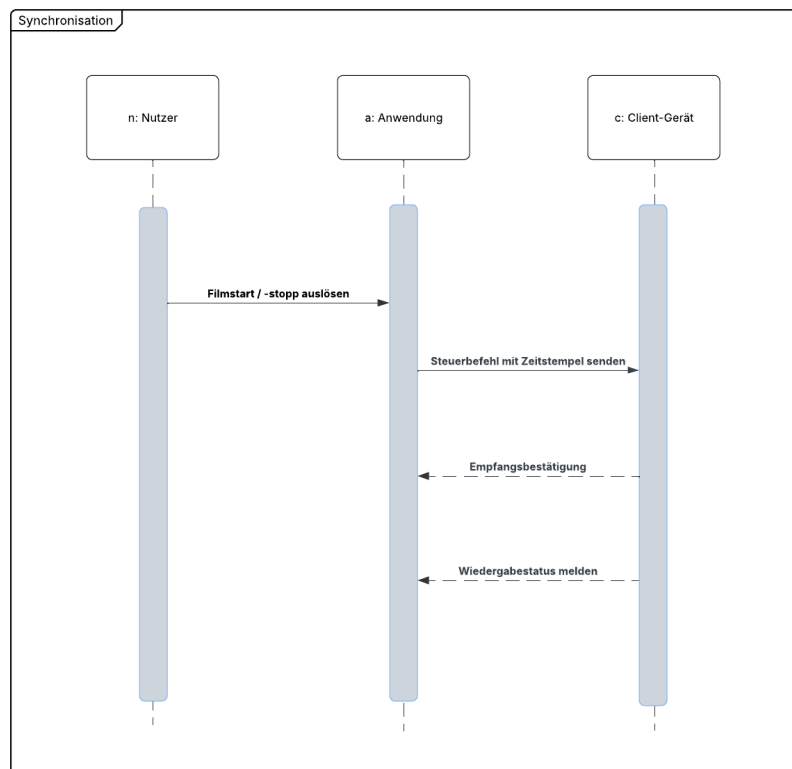


Abbildung 2.8: Multiplayer-Synchronisation (gleichzeitiger Filmstart/-stopp)

2.9 Analyse von Funktionalität <F70>: <Verbindungsaufbau und Management über WLAN>

Die Funktion <F70> ermöglicht die Verbindung aller Geräte über ein gemeinsames WLAN-Netzwerk.

Nach dem Start erkennen sich die Geräte im lokalen Netz automatisch. Sobald eine Verbindung hergestellt ist, tauschen die Geräte relevante Statusdaten aus. Das System erkennt neue Teilnehmer:innen oder Verbindungsverluste und reagiert entsprechend.

Bei einer kurzzeitigen Unterbrechung des WLANs versucht das System, die Verbindung selbstständig wiederherzustellen, ohne dass die Sitzung abgebrochen werden muss.

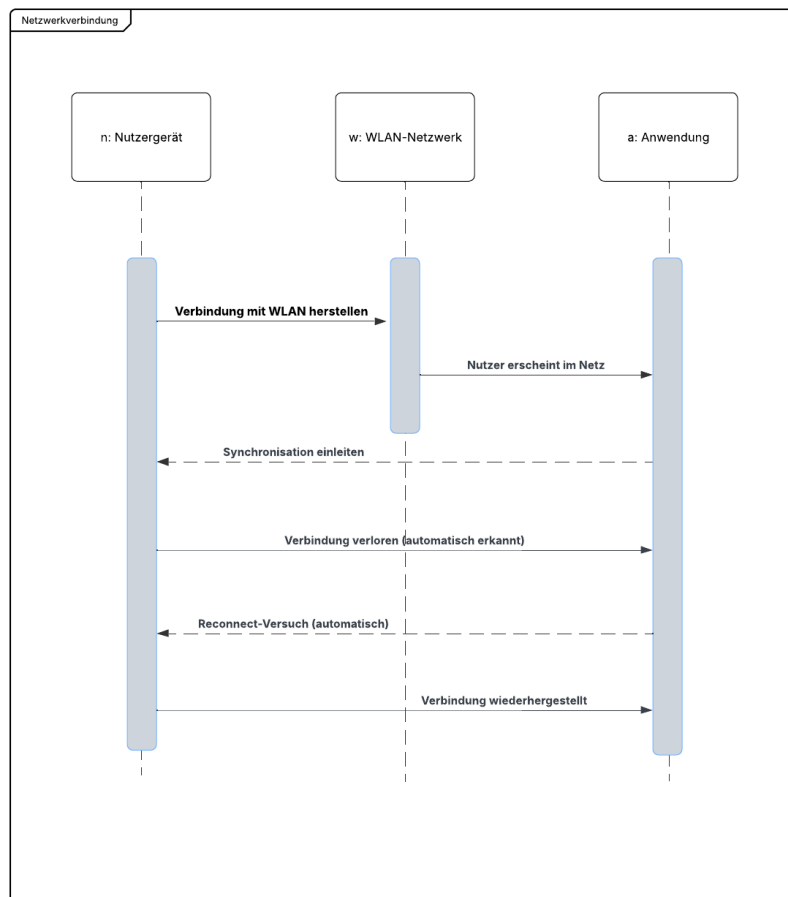


Abbildung 2.9: Verbindungsaufbau und Management über WLAN

3 Resultierende Softwarearchitektur

In diesem Abschnitt wird die Architektur der Anwendung „MR Kino mit Freunden“ beschrieben. Die Software basiert auf einer verteilten Systemarchitektur, bei der mehrere autonome Meta Quest 3 Geräte über ein gemeinsames WLAN-Netzwerk miteinander kommunizieren. Eine zentrale Rolle übernimmt das Gerät mit der niedrigsten IP-Adresse, das automatisch zum Host gewählt wird. Die übrigen Geräte verhalten sich als Clients.

Die Architektur ermöglicht die synchrone Wiedergabe eines Films in Mixed Reality sowie eine koordinierte Benutzerinteraktion über Netzwerkkommunikation. Für die Synchronisation der Geräte kommen die Protokolle UDP (für die Host-Erkennung), ENet (für Multiplayer-Kommunikation und RPCs) sowie HTTP (für den Videoabruf vom Server) zum Einsatz. Die Anwendung wurde mit der Godot Engine in Kombination mit OpenXR entwickelt.

3.1 Komponentenspezifikation

Die Software besteht aus mehreren klar abgegrenzten Komponenten, die jeweils eine spezifische Aufgabe innerhalb der Anwendung übernehmen. In Abbildung 3.1 ist die Struktur dieser Komponenten und ihre Beziehungen zueinander dargestellt.

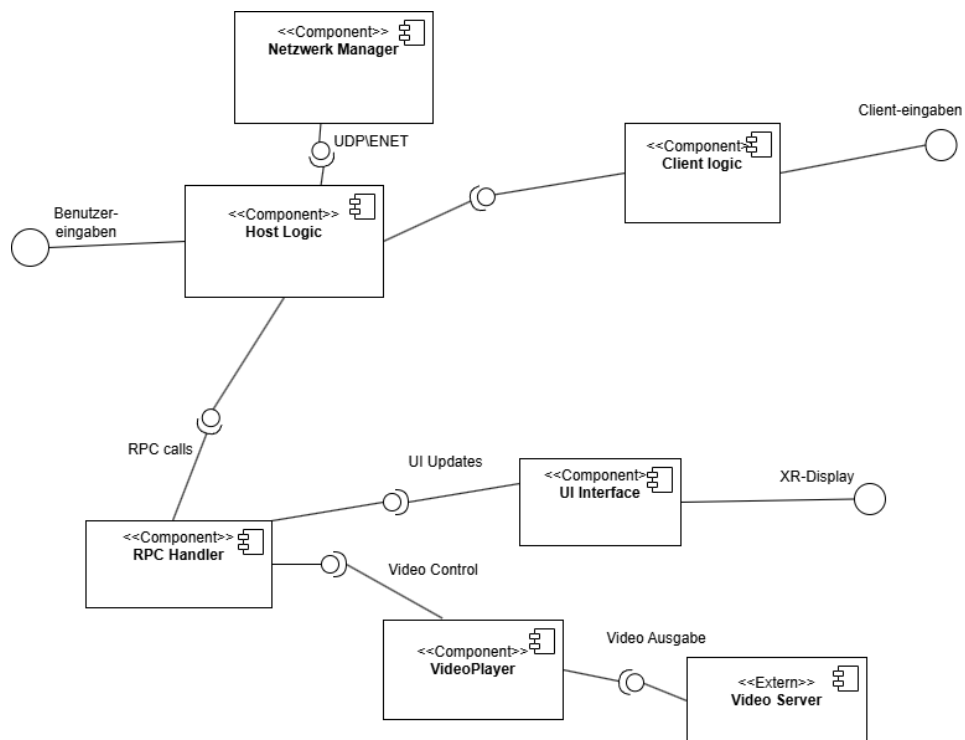


Abbildung 3.1: Komponentendiagramm

Komponente $\langle C10 \rangle$: Host Logic

Diese Komponente ist nur auf dem Host aktiv und zuständig für die Verwaltung des Abstimmungsprozesses, die Berechnung des Abstimmungsergebnisses und den Start der synchronen Wiedergabe auf allen Geräten.

Komponente $\langle C20 \rangle$: Client Logic

Verwaltet alle clientseitigen Interaktionen. Empfängt RPC-Befehle vom Host, aktualisiert lokale Zustände und übernimmt die Steuerung der Benutzereingaben.

Komponente $\langle C30 \rangle$: RPC Handler

Empfängt und verarbeitet Remote Procedure Calls (z.B. Play, Pause, Szene wechseln), um eine synchrone Wiedergabe auf allen Clients zu gewährleisten.

Komponente $\langle C40 \rangle$: User Interface

Stellt die grafische Benutzeroberfläche für die Eingabe von Namen, die Auswahl von Filmszenen sowie die Abstimmung zur Verfügung. Die UI ist vollständig in XR umgesetzt.

Komponente $\langle C50 \rangle$: Video Player

Verantwortlich für das Abspielen des Videos auf dem jeweiligen Gerät. Die Videodateien werden über HTTP vom zentralen Video-Server geladen oder gestreamt. Die Steuerung erfolgt synchronisiert per RPCs.

Komponente $\langle C60 \rangle$: Netzwerkmanager

Erkennt andere Geräte im WLAN, bestimmt den Host anhand der niedrigsten IP-Adresse und initialisiert die ENet-Verbindung. Diese Komponente ist auf allen Geräten aktiv.

Komponente $\langle C70 \rangle$: Video Server (extern)

Ein zentraler HTTP-Server, der die Videodateien bereitstellt. Jedes Gerät greift selbstständig auf diesen Server zu, um die gewählte Szene lokal wiederzugeben.

3.2 Schnittstellenspezifikation

In diesem Abschnitt werden die Schnittstellen beschrieben, über die die einzelnen Komponenten miteinander kommunizieren. Die Schnittstellen sind den jeweiligen Aufgaben und Kommunikationsprotokollen (UDP, ENet, HTTP) zugeordnet und ermöglichen die Interaktion zwischen Host, Clients und dem externen Video-Server.

Schnittstelle $\langle I10 \rangle$: Host-Erkennung (UDP)

Operation	Beschreibung
IP-AdressenScannen()	Erkennt alle Geräte im lokalen Netzwerk über UDP-Broadcast.
NiedrigsteIPWählen()	Vergleicht die IP-Adressen und bestimmt das Gerät mit der niedrigsten Adresse als Host.

Schnittstelle $\langle I20 \rangle$: Multiplayer-Kommunikation (ENet/RPC)

Operation	Beschreibung
SzenenvorschlägeEmpfangen()	Clients senden ihre gewählte Szene an den Host.
AbstimmungErgebnisSenden()	Der Host sendet das Abstimmungsergebnis an alle Clients.
VideoSteuerungRPC()	Der Host sendet Play, Pause oder Seek-Befehle per Remote Procedure Call.

Schnittstelle $\langle I30 \rangle$: Benutzeroberfläche (UI)

Operation	Beschreibung
NameEingeben()	Nutzer:innen geben über die UI ihren Namen ein.
SzeneWählen()	UI-Komponente ermöglicht die Auswahl einer Filmszene.
Abstimmen()	Abgabe der Stimme über die Benutzeroberfläche.
AnzeigeErgebnis()	Darstellung des Abstimmungsergebnisses und der geladenen Szene.

Schnittstelle $\langle I40 \rangle$: Videowiedergabe (HTTP + Player)

Operation	Beschreibung
VideoAnfordern()	Der Video Player fordert die Videodatei vom HTTP-Server an.
VideoAbspielen()	Startet die Wiedergabe des geladenen Videos.
VideoPausieren()	Pausiert die laufende Wiedergabe.
ZuPositionSpringen()	Springt zu einer bestimmten Stelle im Video (Seek).

Schnittstelle $\langle I50 \rangle$: Verbindung zum Video-Server (HTTP)

Operation	Beschreibung
HTTPVerbindungHerstellen()	Verbindet sich mit dem externen Server über HTTP.
VideoDateiLaden()	Lädt die gewünschte Videodatei für die Wiedergabe.

3.3 Protokolle für die Benutzung der Komponenten

In dem folgenden Abschnitt wird für jede Komponente festgehalten, wie diese korrekt zu verwenden ist. Dazu gibt es zu jeder Komponente eine Protokoll-Statechart, welche den korrekten Umgang beschreibt, und einen erläuternden Text. Weiterhin wird für jede Komponente mögliche Wiederverwendbarkeit bewertet. So ist es möglich, dass Teile der Software für andere Projekte sinnvoll übernommen werden können, um so Arbeitsaufwand und mögliche Entwicklungskosten zu schonen.

3.3.1 Protokoll für <C10>: Host Logic

Die Host-Komponente befindet sich zu Beginn im Zustand *Warten auf Spieler*, in dem auf die Verbindung aller Clients gewartet wird. Sobald alle Spieler verbunden sind, wechselt der Host in den Zustand *Voting starten* und sendet einen RPC-voting-Befehl an die Clients. Daraufhin sammelt der Host im Zustand *Votes sammeln* die eingehenden Stimmen. Sobald alle Stimmen vorliegen, wird im Zustand *Ergebnis berechnen* der Gewinner ermittelt und im Anschluss im Zustand *Ergebnis senden* an alle Clients verteilt. Nach erfolgreicher Verteilung übernimmt der Host im Zustand *Wiedergabe steuern* die Kontrolle über die Medienwiedergabe. Diese endet entweder regulär mit dem Zustand *Film zu Ende*, oder der Ablauf wird in einem der Zwischenschritte durch einen Fehler unterbrochen, woraufhin der Zustand *Fehler* erreicht wird. Das Diagramm bildet somit eine vollständige Sitzung von Spielvorbereitung bis zur Mediensteuerung ab, inklusive Fehlerbehandlung.

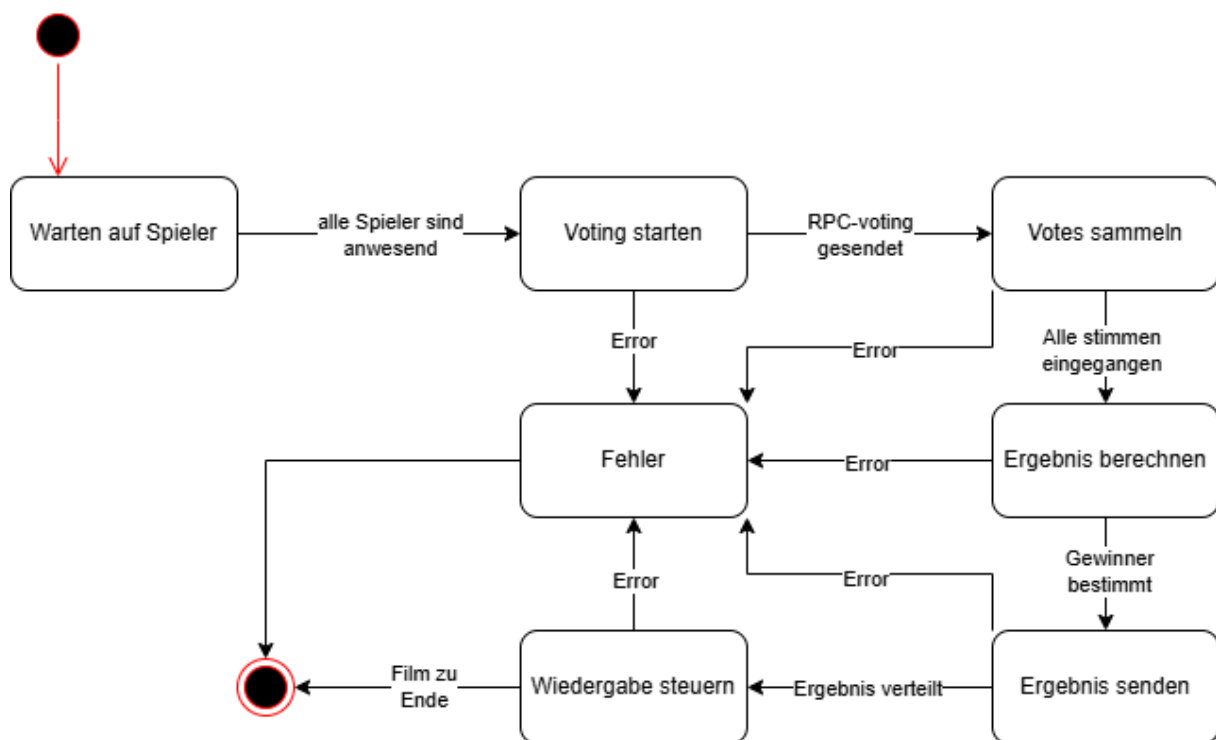


Abbildung 3.2: State-Chart der Host Logic-Komponente

3.3.2 Protokoll für <C20>: Client Logic

Die Client-Komponente befindet sich zunächst im Zustand *Bereit* und wartet auf Anweisungen vom Host. Sobald der Host das Signal zum Start einer Abstimmung gibt, wechselt der Client in den Zustand *Voting aktiv*, in dem der Benutzer eine Szene oder Option auswählen kann. Nach der Stimmabgabe befindet sich der Client im Zustand *Warte auf Ergebnis*, bis das endgültige Abstimmungsergebnis empfangen wird. Mit dem Ergebnis tritt der Client wieder in den Zustand *Bereit* über und erwartet das Startsignal zur Wiedergabe. Im Zustand *Wiedergabe* wird das Video abgespielt, wobei Pausieren und Fortsetzen über die Zustände *Pausiert* und zurück zu *Wiedergabe* geregelt werden. Erkennt der Client eine zeitliche Abweichung, wird über den Zustand *Synchronisieren* eine Anpassung vorgenommen, bevor die reguläre Wiedergabe fortgesetzt wird. Erkennt der Client eine zeitliche Abweichung, wird über den Zustand *Synchronisieren* eine Anpassung vorgenommen, bevor die reguläre Wiedergabe fortgesetzt wird.

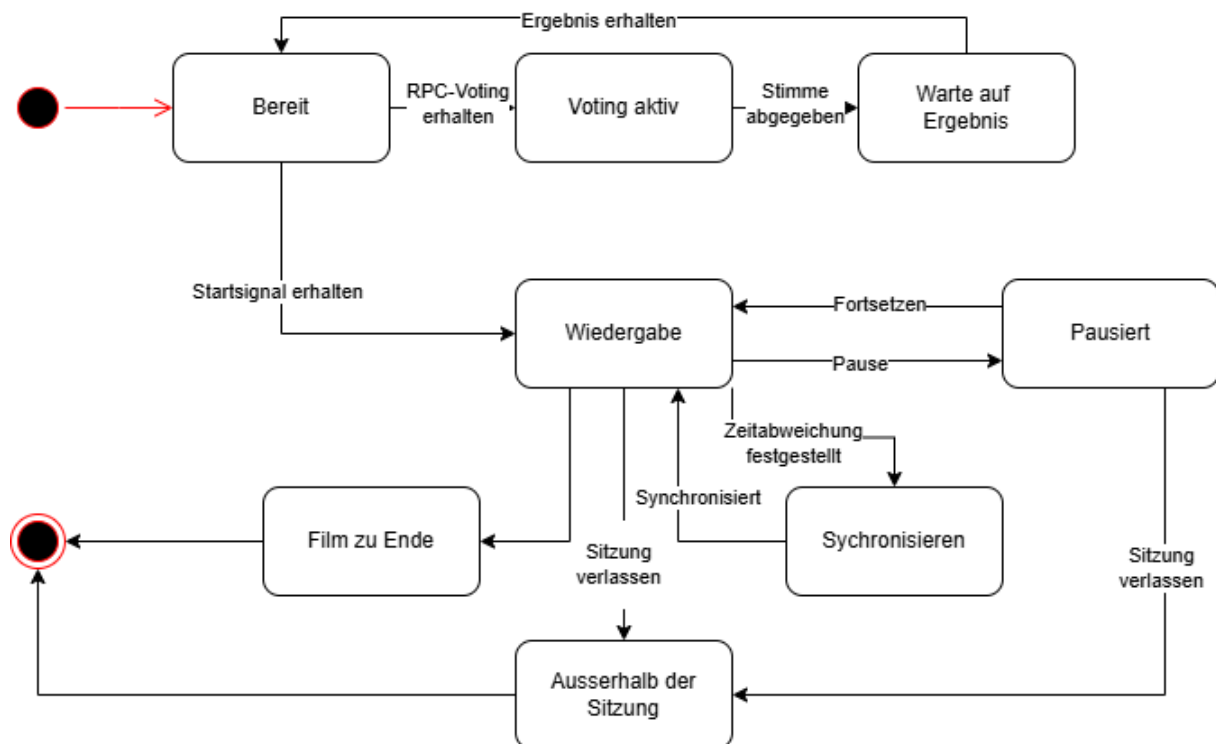


Abbildung 3.3: State-Chart der Client Logic-Komponente

3.3.3 Protokoll für <C30>: RPC Handler

Die RPC-Verarbeitungskomponente des Clients befindet sich standardmäßig im Zustand *Warten auf RPC*, in dem sie kontinuierlich auf vom Host gesendete Kommandos lauscht. Erhält der Client ein Play-Kommando, wechselt er in den Zustand *Verarbeite play* und übergibt den Abspielbefehl an die Videokomponente. Beim Empfang eines Pause-RPC erfolgt entsprechend der Übergang in den Zustand *Verarbeite pause*, woraufhin die Wiedergabe pausiert wird. Der Zustand *Verarbeite voting start* wird aktiviert, wenn der Host den Start einer Abstimmung einleitet; in diesem Fall wird die Abstimmung an die Client-Logik weitergereicht. Schließlich verarbeitet der Client im Zustand *Verarbeite voting Ergebnis* das vom Host gesendete Abstimmungsergebnis, z. B. die gewählte Szene. Nach Ausführung jeder Aktion kehrt die Komponente wieder in den Ausgangszustand *Warten auf RPC* zurück, um neue Anfragen zu empfangen.

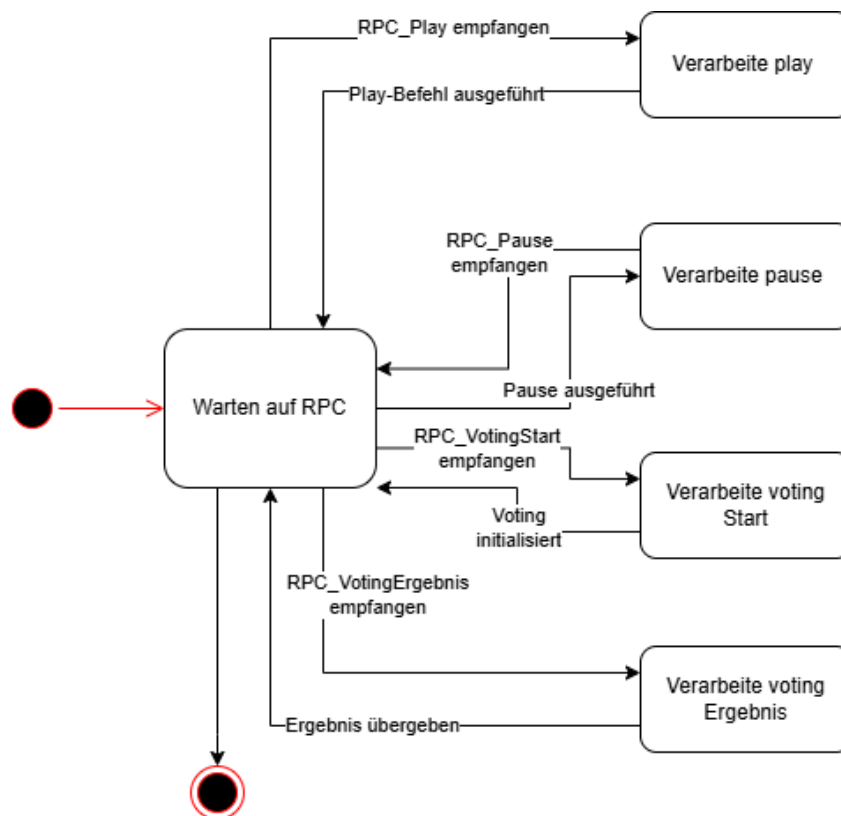


Abbildung 3.4: State-Chart der RPC-Komponente

3.3.4 Protokoll für <C40>: User Interface

Die Benutzerinterface-Komponente beginnt im Zustand *Name Eingabe aktiv*, in dem der Benutzer über eine XR-Tastatur seinen Namen eingibt. Nach Bestätigung des Namens wechselt die Komponente in den Zustand *Szene Auswahl aktiv*, in dem eine Auswahl an Szenen zur Verfügung steht. Sobald eine Szene ausgewählt wurde, geht das Interface in den Zustand *Warte auf Voting Start* über, in dem es auf das Startsignal für die Abstimmung vom Host wartet. Nach Empfang dieses Signals wird der Zustand *Voting aktiv* erreicht, in dem der Benutzer seine Stimme abgeben kann. Mit Abschluss der Abstimmung wechselt das Interface in den Zustand *Voting abgeschlossen*, in dem keine weitere Eingabe möglich ist und z. B. ein „Warte auf Ergebnis“-Hinweis angezeigt wird. Sobald die Wiedergabe beginnt, wird die UI im Zustand *UI Versteckt* ausgeblendet oder minimiert, um die Sicht auf den Medieninhalt freizugeben.

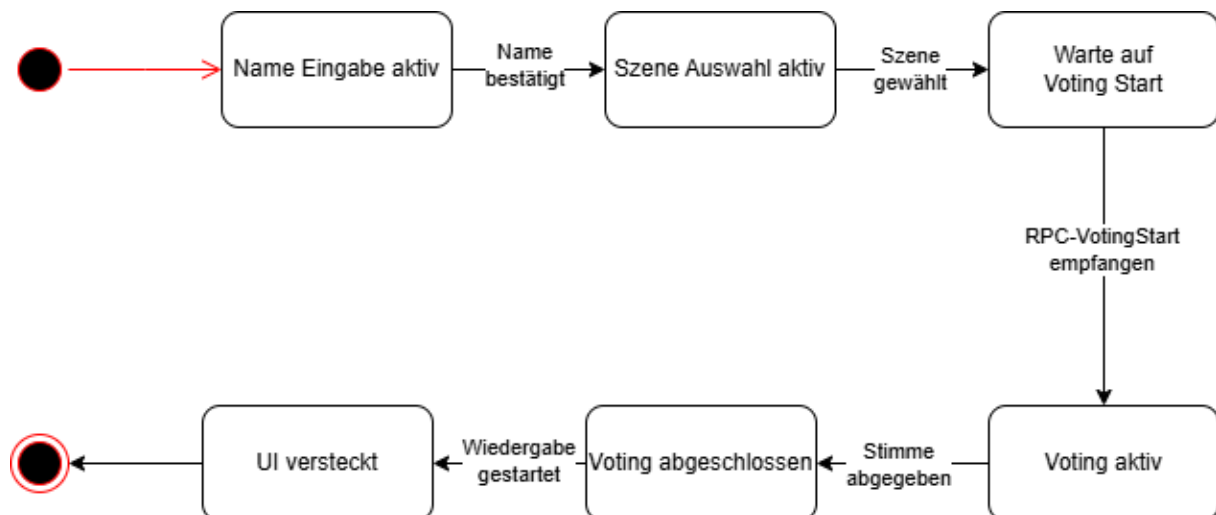


Abbildung 3.5: State-Chart der UI-Komponente

3.3.5 Protokoll für <C50>: Video Player

Die Videoplayer-Komponente befindet sich nach dem Start im Zustand *Bereit*, in dem sie auf einen Befehl zum Starten der Wiedergabe wartet. Wird dieser Befehl ausgeführt, fordert die Komponente im Zustand *Chunks anfordern* drei aufeinanderfolgende Videosegmente vom Server an. Nach erfolgreichem Download werden diese im Zustand *Pufferung* zwischengespeichert, bis der Puffer ausreichend gefüllt ist. Anschließend beginnt im Zustand *Abspielen* die Wiedergabe. Sobald das Ende der gepufferten Chunks erreicht ist, wechselt die Komponente in den Zustand *Warten auf den nächsten Chunk*, um weitere Segmente nachzuladen. Zusätzlich kann der Benutzer die Wiedergabe manuell unterbrechen (*Pausieren*) oder vollständig stoppen (*Anhalten*). Bei einem auftretenden Fehler, wie z.B. einem fehlgeschlagenen Chunk-Download, geht die Komponente in den Zustand Fehler über. Die Komponente ist speziell auf das asynchrone Chunk-basierte Streaming über HTTP ausgelegt und ohne die serverseitige Unterstützung nicht funktionsfähig.

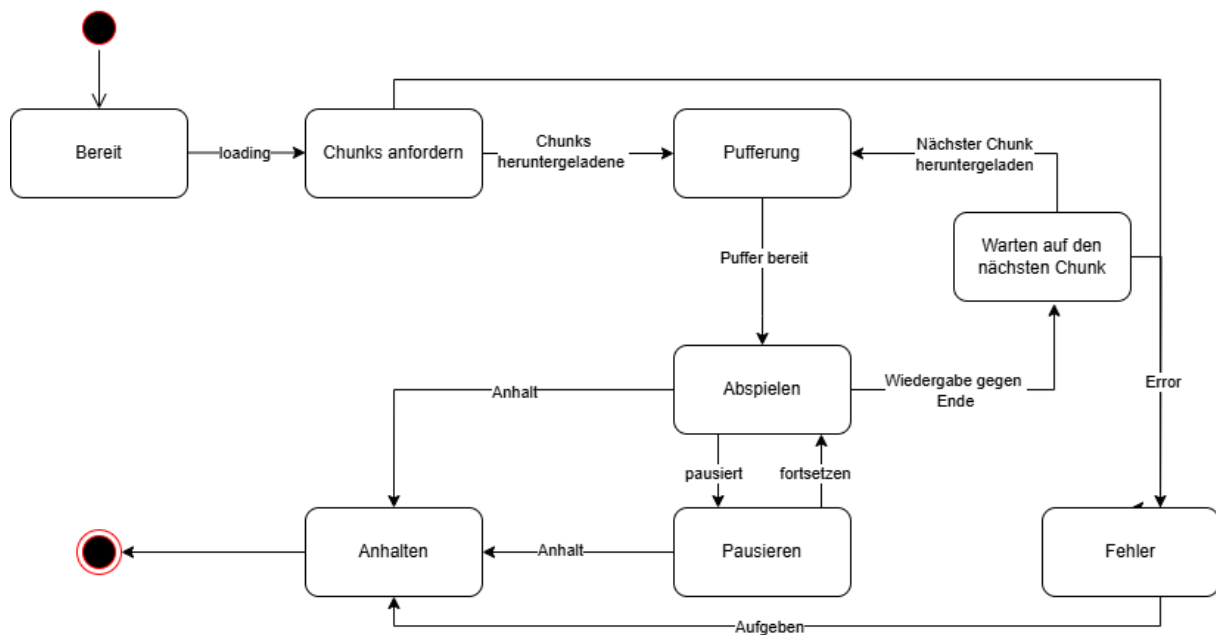


Abbildung 3.6: State-Chart der Video Player-Komponente

3.3.6 Protokoll für <C60>: Netzwerk Manager

Die Komponente Netzwerkmanager ist auf allen Geräten aktiv und übernimmt die automatische Einrichtung des Kommunikationsnetzwerks. Im Zustand *StarteScan* wird nach anderen Geräten im WLAN gesucht. Sobald Teilnehmer erkannt wurden (*Teilnehmer erkennen*), bestimmt die Komponente im nächsten Schritt (*Host bestimmen*) die Rolle des Hosts anhand der niedrigsten IP-Adresse im Netzwerk. Anschließend wird im Zustand *ENet initialisieren* die ENet-Verbindung vorbereitet und entsprechend der zugewiesenen Rolle konfiguriert. Bei erfolgreichem Verbindungsaufbau wechselt der Netzwerkmanager in den Zustand *Verbindung herstellen* und ermöglicht damit die weitere Kommunikation zwischen den Geräten. Tritt in einem der Schritte ein Fehler auf – etwa bei der IP-Erkennung oder dem Verbindungsaufbau –, wird in den Zustand Fehler übergegangen.

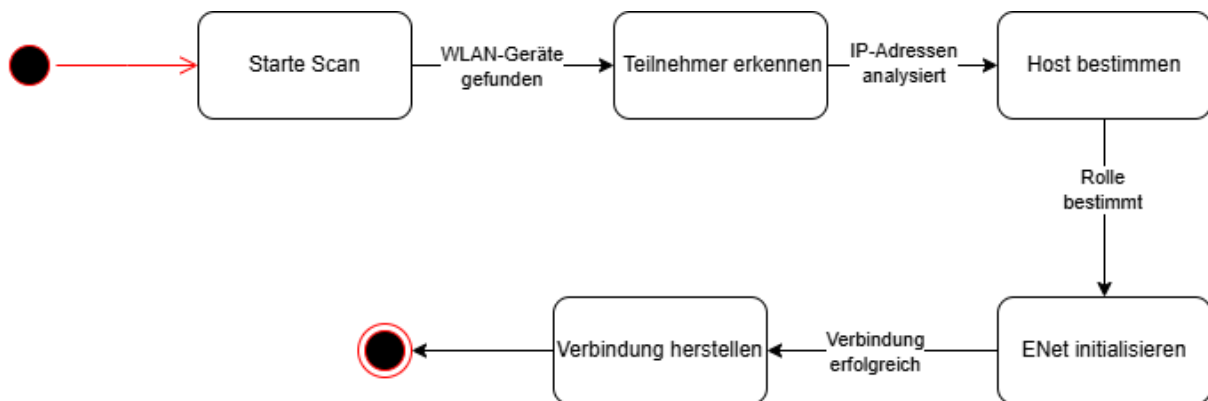


Abbildung 3.7: State-Chart der Netzwerk Manager-Komponente

3.3.7 Protokoll für <C70>: Video Server (extern)

Die Server-Komponente dient dem Ausliefern einzelner Video-Chunks über HTTP und arbeitet nach einem einfachen, zustandsbasierten Ablauf. Nach dem Zustand *Bereit*, in dem auf eingehende Anfragen gewartet wird, wechselt der Server bei einer gültigen Anfrage in den Zustand *Empfangene Anfrage* und anschließend in *Chunk ausliefern*, um die angeforderte Datei bereitzustellen. Falls ein Fehler auftritt, etwa bei einer ungültigen Anfrage oder fehlender Datei, wird stattdessen der Zustand *Fehler* erreicht. Nach Abschluss des Vorgangs kehrt der Server in den Ausgangszustand zurück oder beendet den aktuellen Anfragezyklus. Da jede Chunk-Anfrage unabhängig behandelt wird, bleibt der Server im Dauerbetrieb reaktiv gegenüber neuen Verbindungen.

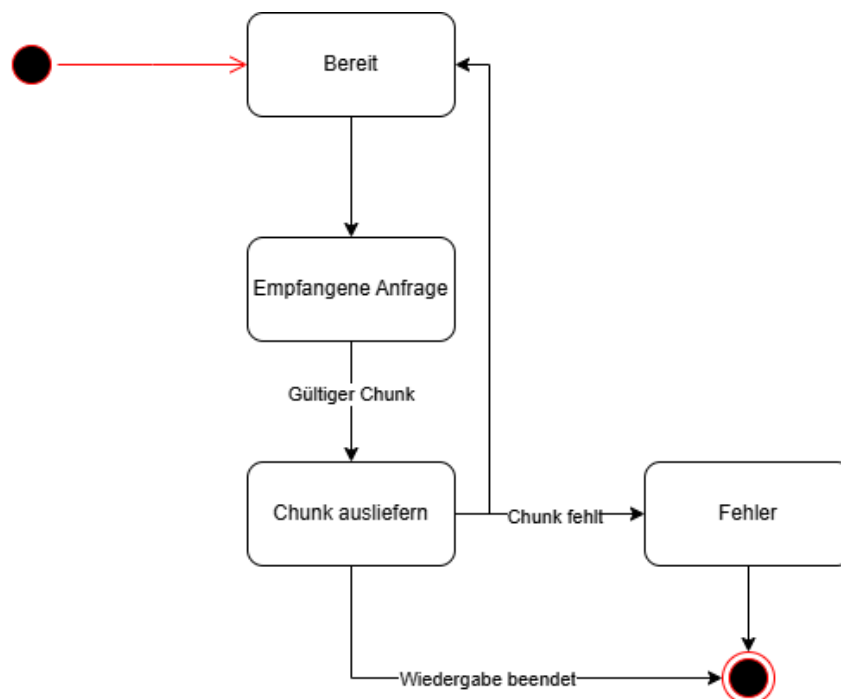


Abbildung 3.8: State-Chart der Server-Komponente

4 Verteilungsentwurf

Das Mixed-Reality-Kino-System läuft vollständig und lokal auf Meta Quest 3 Headsets. Beim Start verbindet sich jedes Gerät mit dem WLAN und das Gerät mit der niedrigsten IP-Adresse wird automatisch zum Host. Dieser zählt die Stimmen, bestimmt die gewählte Szene und synchronisiert die Wiedergabe per RPCs an alle Clients. Alle Headsets verfügen über eine Benutzeroberfläche zur Namenseingabe, Szenenwahl und Abstimmung. Nach der Wahl wird die Szene geladen und die Wiedergabe synchron gestartet. Die Videodaten stammen von einem zentralen HTTP-Server, auf den alle Geräte selbstständig zugreifen. Die Kommunikation erfolgt über UDP (Host-Erkennung) und ENet (Multiplayer). Die Anwendung wurde mit Godot und OpenXR speziell für die Meta Quest entwickelt. Die Architektur ermöglicht ein vollständig kabelloses, eigenständiges und synchrones Kinoerlebnis.

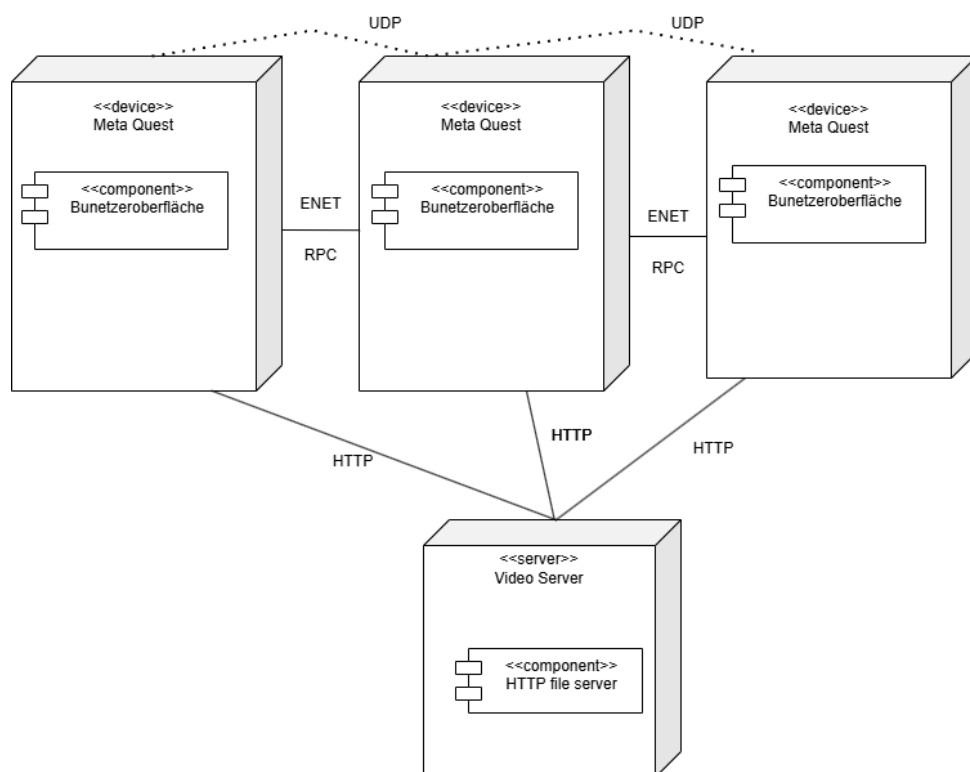


Abbildung 4.1: Verteilungsdiagramm

5 Implementierungsentwurf

In diesem Kapitel wird die Implementierung der im Fach- und Softwarearchitekturentwurf beschriebenen Komponenten dargestellt. Jede Teilkomponente wird einzeln beschrieben, inklusive eines Klassendiagramms und einer Erläuterung zur jeweiligen Struktur und Rolle innerhalb des Gesamtsystems.

5.1 Implementierung von Komponente <C10>: <SessionManager>

Der SessionManager steuert die zentrale Sitzung der Anwendung. Er übernimmt die Rollenvergabe (z.B. Host), koordiniert die Synchronisation der Teilnehmer:innen und steuert den Ablaufstatus der Sitzung. Die Kommunikation erfolgt hauptsächlich über den NetworkManager. Die Komponente ist als Singleton (Autoload) umgesetzt und nutzt Signale sowie einfache Zustände (INIT, ACTIVE, ENDED).

5.1.1 Paket-/Klassendiagramm

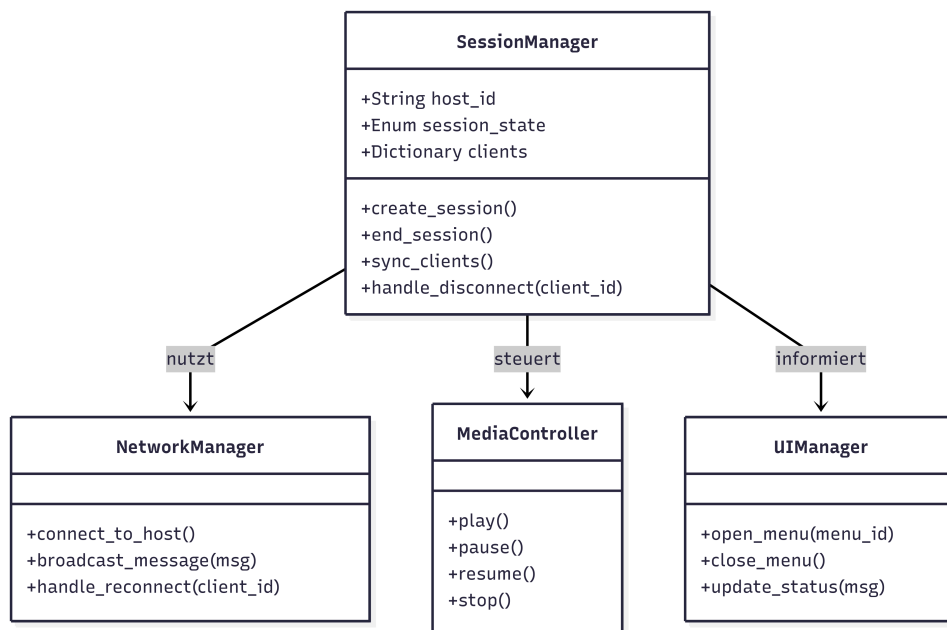


Abbildung 5.1: Klassendiagramm für Komponente <C10>: SessionManager

5.1.2 Erläuterung

Die Komponente SessionManager ist für die Verwaltung und den Ablauf einer synchronisierten Sitzung verantwortlich. Sie ist als Singleton (Autoload) implementiert und befindet sich stets im globalen Speicherraum der Anwendung.

Attribute:

- `host_id`: `String` – ID des Hosts
- `session_state`: `Enum` – Status der Sitzung (z. B. INIT, ACTIVE, ENDED)
- `clients`: `Dictionary` – Liste verbundener Clients

Methoden:

- `create_session()` – Initialisiert eine neue Sitzung
- `end_session()` – Beendet die aktuelle Sitzung
- `sync_clients()` – Führt die Synchronisation zwischen Teilnehmern aus
- `handle_disconnect(client_id)` – Entfernt Client aus aktiver Sitzung

Kommunikationspartner: NetworkManager, MediaController, UIManager

5.2 Implementierung von Komponente <C20>: <MediaController>

Der MediaController ist für die Steuerung der Wiedergabe zuständig. Er verarbeitet zentrale Befehle wie Play, Pause und Stop und sorgt dafür, dass alle Clients synchron abspielen. Die Steuerbefehle erhält er zumeist vom SessionManager und leitet den Status ggf. an UI und Netzwerk weiter.

5.2.1 Paket-/Klassendiagramm

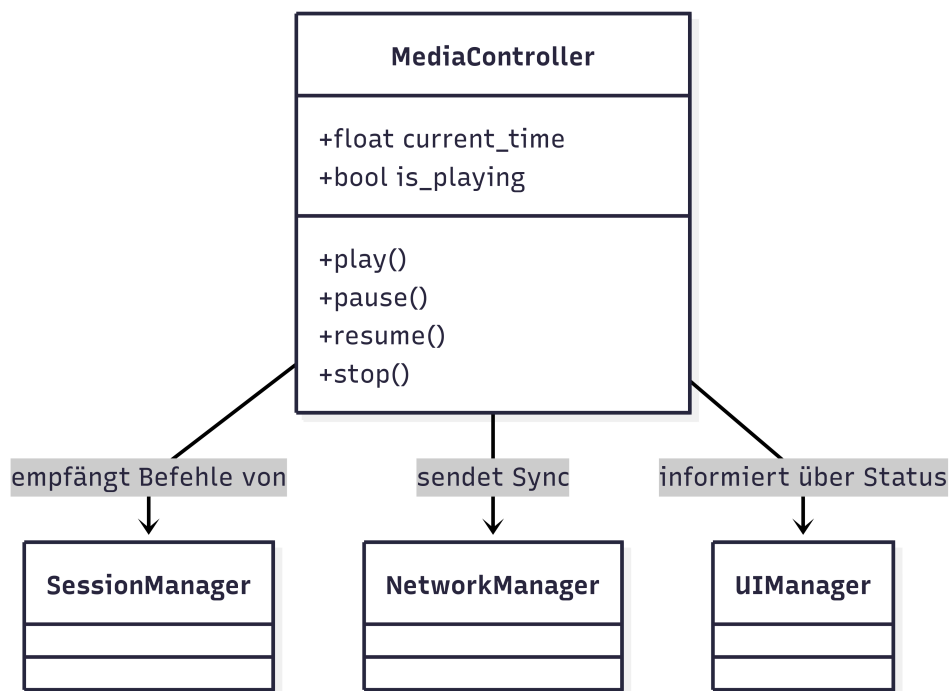


Abbildung 5.2: Klassendiagramm für Komponente <C20>: MediaController

5.2.2 Erläuterung

Die Komponente MediaController ist zuständig für die Steuerung der Filmwiedergabe, insbesondere das Starten, Pausieren, Fortsetzen und Stoppen des Films.

Attribute:

- `current_time: float` – aktueller Wiedergabezeitpunkt
- `is_playing: bool` – Status der Wiedergabe

Methoden:

- `play()` – Startet den Film
- `pause()` – Pausiert den Film
- `resume()` – Setzt die Wiedergabe fort
- `stop()` – Stoppt die Wiedergabe und setzt Zeit auf 0

Kommunikationspartner: SessionManager, NetworkManager, UIManager

5.3 Implementierung von Komponente <C30>: <UIManager>

Der UIManager steuert die Anzeige von Menüs und Benutzerinteraktionen in der MR-Umgebung. Er stellt sicher, dass Overlays wie Play/Pause, Filmauswahl oder Statusmeldungen korrekt dargestellt und bei Bedarf ein- oder ausgeblendet werden. Die Komponente ist eng mit dem Media-Controller und EnvironmentLoader verknüpft.

5.3.1 Paket-/Klassendiagramm

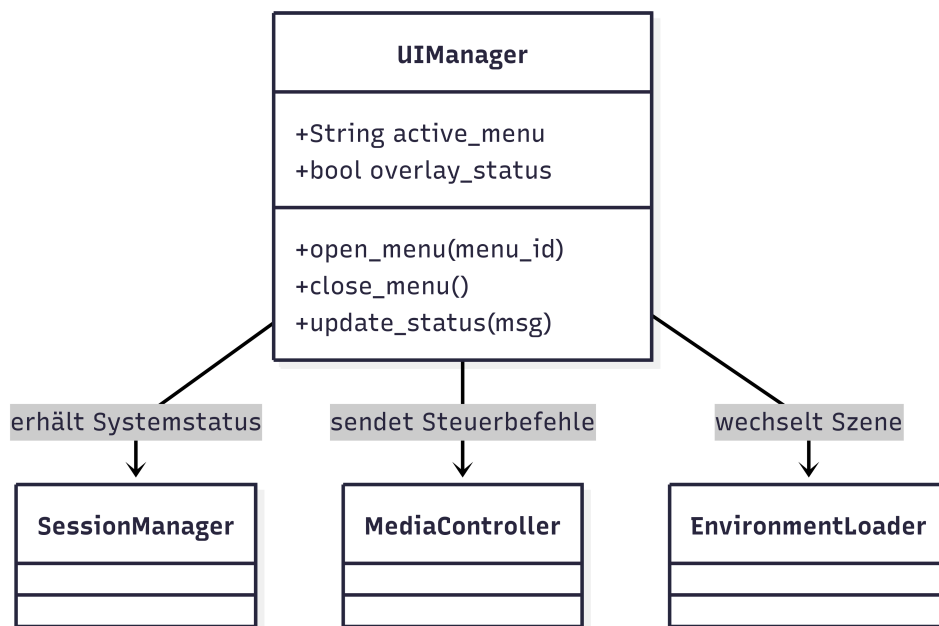


Abbildung 5.3: Klassendiagramm für Komponente <C30>: UIManager

5.3.2 Erläuterung

Die Komponente UIManager ist für die visuelle Darstellung und Benutzerinteraktion innerhalb der MR-Anwendung zuständig. Sie stellt Menüs, Schaltflächen und Feedback an die Nutzer:innen bereit.

Attribute:

- `active_menu: String` – aktuell geöffnetes UI-Menü
- `overlay_status: bool` – Sichtbarkeit des UI in MR

Methoden:

- `open_menu(menu_id)` – Öffnet ein spezifisches Menü
- `close_menu()` – Schließt aktuell sichtbares Menü
- `update_status(msg)` – Zeigt Systemstatus als Overlay

Kommunikationspartner: SessionManager, MediaController, EnvironmentLoader

5.4 Implementierung von Komponente <C40>: <EnvironmentLoader>

Der EnvironmentLoader ist für das Laden und Umschalten der virtuellen Szenen verantwortlich. Je nach Zustand der Anwendung (Start, Wiedergabe, Beenden) werden passende MR-Umgebungen wie Wohnzimmer oder Kinosaal geladen. Die Steuerung erfolgt meist durch den UIManager.

5.4.1 Paket-/Klassendiagramm

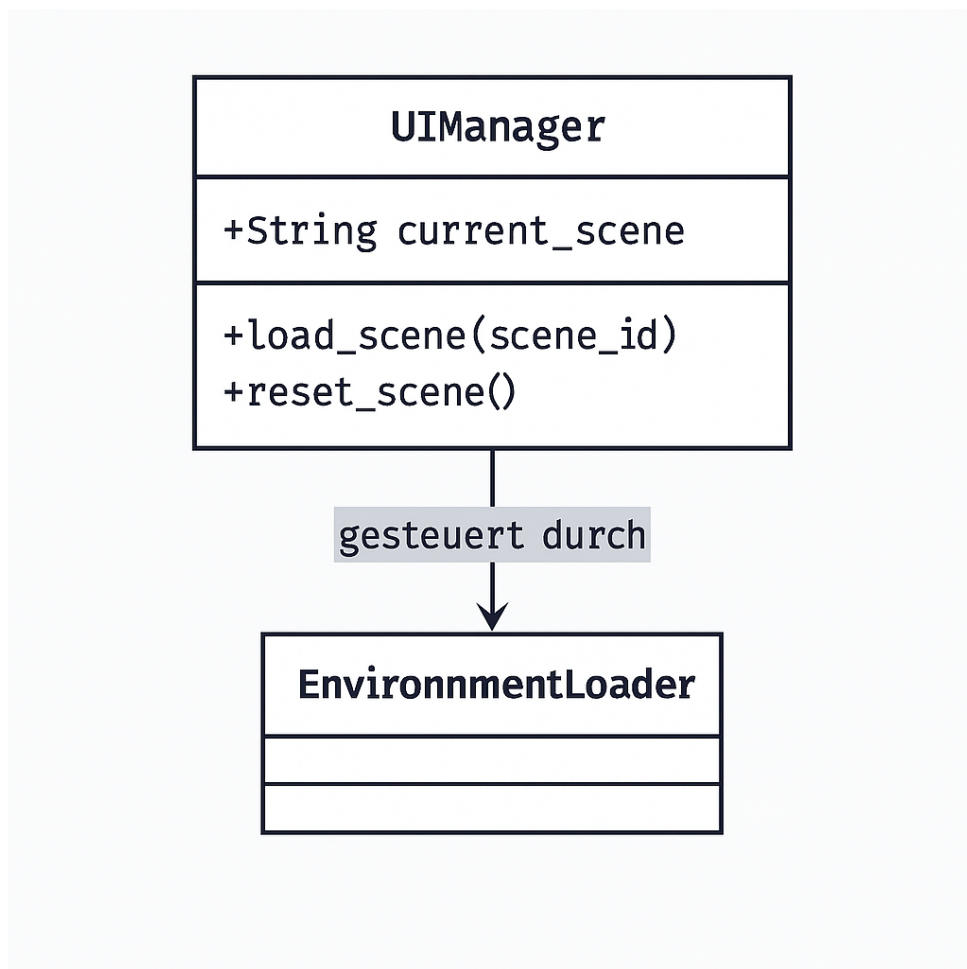


Abbildung 5.4: Klassendiagramm für Komponente <C40>: EnvironmentLoader

5.4.2 Erläuterung

Der EnvironmentLoader ist für das Laden und Wechseln der virtuellen Szenen zuständig (Wohnzimmer, Kinosaal usw.).

Attribute:

- `current_scene: String` – aktuell aktive Szene

Methoden:

- `load_scene(scene_id)` – Wechselt die Szene basierend auf dem Identifier
- `reset_scene()` – Setzt die aktuelle Szene zurück

Kommunikationspartner: UIManager

5.5 Implementierung von Komponente <C50>: <NetworkManager>

Der NetworkManager koordiniert die Netzwerkkommunikation zwischen Host und Clients. Er erkennt neue Teilnehmer:innen, verwaltet Verbindungsabbrüche und überträgt Steuerbefehle wie Start, Pause oder Synchronisierungsdaten. Die Komponente arbeitet eng mit SessionManager und MediaController zusammen.

5.5.1 Paket-/Klassendiagramm

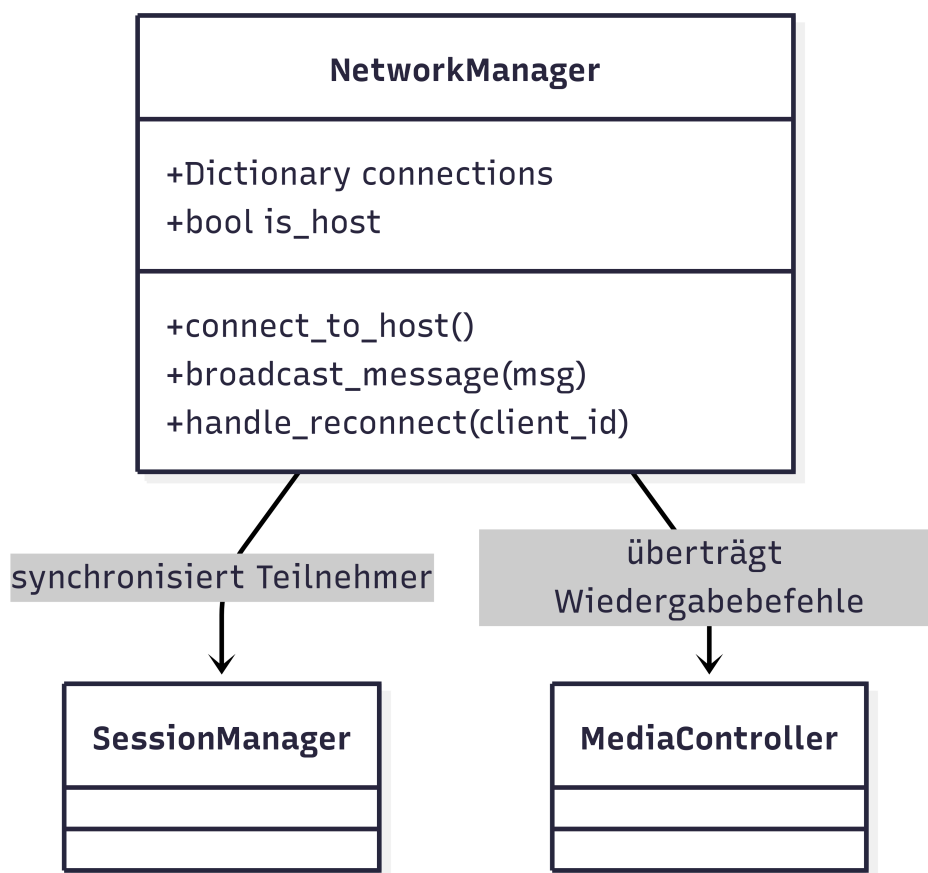


Abbildung 5.5: Klassendiagramm für Komponente <C50>: NetworkManager

5.5.2 Erläuterung

Der NetworkManager koordiniert die Kommunikation zwischen mehreren Clients. Er erkennt neue Clients, verwaltet Verbindungsabbrüche und verteilt Steuerinformationen.

Attribute:

- `connections`: Dictionary – aktive Netzwerkverbindungen
- `is_host`: bool – Ist dieser Client der Host?

Methoden:

- `connect_to_host()` – Stellt Verbindung zum Host her
- `broadcast_message(msg)` – Versendet Nachricht an alle Clients
- `handle_reconnect(client_id)` – Reconnect-Prozess bei Ausfall

Kommunikationspartner: SessionManager, MediaController

6 Datenmodell

Unsere Anwendung empfängt eingehende Daten, die dann formatiert und weiterverarbeitet werden. Dieses Kapitel dient der Erläuterung des Datenmodells, um aufzuzeigen, wie diese Daten innerhalb unserer Anwendung genutzt werden. Die Datenverarbeitung stellt einen der größten und entscheidendsten Aspekte unserer Arbeit dar, weshalb ein reibungsloser und präziser Ablauf von höchster Bedeutung ist.

6.1 Diagramm

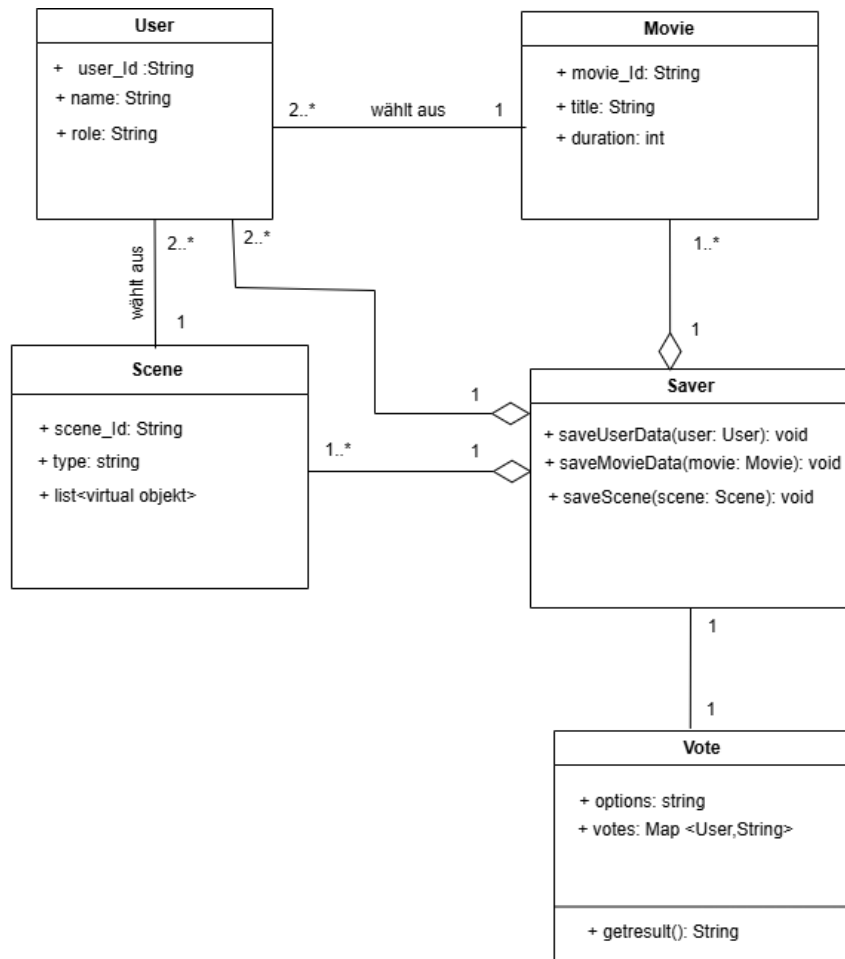


Abbildung 6.1: Klassendiagramm

6.2 Erläuterung

Die folgenden Tabelleneinträge dienen dazu, die Entitäten im obigen Klassendiagramm weiter zu erläutern und diese verständlich darzustellen.

Benutzer $\langle E10 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Movie	1	Min: 1, Max: 1	Jeder Benutzer gibt genau eine Stimme für genau einen Film ab.
Scene	1	Min: 1, Max: 1	Jeder Benutzer gibt genau eine Stimme für genau eine Szene ab.

Movie $\langle E20 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
User	2..*	Min: 2 , Max: *	Eine Film kann null oder mehrfach von Benutzern abgestimmt werden.
Saver	1	Min: 1 , Max: 1	Jede Filmstimme wird im System gespeichert.

Scene $\langle E30 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
User	2..*	Min:2 , Max: *	Eine Szene kann null- oder mehrfach von Benutzern abgestimmt werden.

Saver	1	Min: 1 , Max: 1	Jede Stimme für einen Szene wird im System gespeichert.
-------	---	-----------------	---

Saver $\langle E40 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
User	2..*	Min: 2 , Max: *	Der Saver empfängt die Benutzerdaten.
Movie	1..*	Min:1 , Max: *	Der Saver empfängt die Filmstimmendaten.
Scene	1..*	Min: , Max:	Der Saver empfängt die szenestimmendaten.

vote $\langle E50 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Saver	1	Min: 1, Max: 1	Das Vote speichert verfügbare Abstimmungsoptionen sowie die Stimmen der Benutzer.

7 Konfiguration

In diesem Kapitel werden alle Konfigurationen beschrieben, die für einen reibungslosen Betrieb der Anwendung auf der Meta Quest 3 in einer Mixed-Reality-Umgebung (MR) erforderlich sind. Die Anwendung wurde mit der Godot Engine (Version 4.4) entwickelt und nutzt das OpenXR-Plugin*, um Mixed-Reality-Funktionen auf dem Headset zu ermöglichen.

Für die Nutzung sind keine manuellen Konfigurationen durch die Nutzer erforderlich. Alle Geräte (Meta Quest 3) und Server müssen lediglich mit demselben lokalen WLAN verbunden sein und die Anwendung starten. Die Geräte erkennen sich automatisch und synchronisieren ihre Daten im Hintergrund untereinander.

Die Filme werden nicht lokal auf den Geräten gespeichert, sondern über einen lokalen HTTP-Server bereitgestellt. Dieser Server hostet die verfügbaren Videodateien (im .ogv-Format) und ist über eine IP-Adresse im Netzwerk erreichbar. Die Anwendung ruft beim Start automatisch die Liste aller verfügbaren Filme vom Server ab und stellt sie im Auswahlménü dar.

Externe Konfigurationsdateien existieren nicht. Stattdessen sind alle grundlegenden Einstellungen – wie Szenen oder UI-Zustände* – direkt in den jeweiligen Godot-Szenendateien (.tscn)* hinterlegt.

Beim Start der Anwendung gibt der Nutzer seinen Namen ein und wählt einen Film sowie eine Szene aus. Diese Angaben werden im System gespeichert und automatisch zwischen allen Geräten synchronisiert. Sobald sich alle Nutzer als „bereit“ markiert haben, erscheint ein neues Menü mit dem Film und der Szene, die per Mehrheitswahl ausgewählt wurden. In diesem Menü kann jeder Nutzer die Wiedergabe manuell starten, indem er den „Start Film“-Button drückt. Erst dann beginnt die gemeinsame Wiedergabe.

Das Menü wird zu Beginn in Mixed Reality angezeigt, die eigentliche Wiedergabe erfolgt dann abhängig von der gewählten Szene – entweder in MR oder vollständig in Virtual Reality.

Die Anwendung ist für den lokalen Einsatz konzipiert. Sie benötigt keinen Internetzugang, sondern nur ein gemeinsames WLAN für die Kommunikation zwischen Geräten und mit dem Filmserver. Der Export auf die Meta Quest 3 erfolgt als .apk-Datei* und kann über SideQuest* oder ADB auf das Headset übertragen werden.

8 Änderungen gegenüber Fachentwurf

Im Verlauf der Implementierung wurden einige Anpassungen gegenüber dem ursprünglichen Fachentwurf notwendig, um die Anwendung auf der Meta Quest 3 stabil und benutzerfreundlich zu gestalten:

Zusätzliche Netzwerkkomponente (Filmserver): Im Fachentwurf wurde noch davon ausgegangen, dass die Filme lokal auf jedem Gerät gespeichert sind. Im finalen Entwurf wurde stattdessen ein zentraler HTTP-Server* eingeführt, der im lokalen Netzwerk gehostet wird. Dieser Server stellt alle verfügbaren Filme im .ogv-Format zur Verfügung. Die Anwendung ruft diese Inhalte dynamisch ab.

Dynamische Filmliste: Ursprünglich war eine feste Auswahl an Filmen vorgesehen (z.B. „Film 1“, „Film 2“). Dies wurde durch eine automatische Generierung der Filmliste ersetzt, die beim Start über eine Anfrage an den Server erzeugt wird. Somit passt sich das Menü dynamisch der tatsächlichen Anzahl und den Namen der Filme auf dem Server an.

9 Erfüllung der Kriterien

Im Folgenden werden kurz die Muss-, Soll- und Kannkriterien aus dem Plichtenheft betrachtet und die Komponenten dargestellt, durch die sie umgesetzt werden.

9.1 Musskriterien

⟨RM1⟩ Die App muss die Echtzeitsynchronisation zwischen allen Nutzern unterstützen.

Dies wird durch die Komponenten <C10> **Host Logic**, <C20> **Client Logic**, <C30> **RPC Handler** und <C60> **Netzwerkmanager** gewährleistet, welche gemeinsam die Kommunikation, Zustandsverteilung und Synchronität zwischen den Geräten steuern.

⟨RM2⟩ Die Anwendung muss ein robustes Abstimmungssystem bereitstellen.

Diese Funktion wird durch die <C10> **Host Logic** zur Stimmenausrwertung sowie die <C40> **UI Interface** zur Benutzerinteraktion umgesetzt.

⟨RM3⟩ Die Plattform muss Mixed-Reality-Karten enthalten, um eine immersive Umgebung bereitzustellen. Dies wird durch die Kombination der Komponenten <C40> **UI Interface**, <C50> **Video Player** und <C20> **Client Logic** realisiert, die gemeinsam für Auswahl, Darstellung und Interaktion mit MR-Szenen verantwortlich sind.

⟨RM4⟩ Die Anwendung muss Mixed-Reality-Funktionen bereitstellen, bei denen virtuelle Objekte mit der realen Umgebung verschmelzen.

Die Umsetzung erfolgt über die <C50> **Video Player** zur Darstellung der Inhalte im physischen Raum sowie die <C40> **UI Interface**, die eine stabile Einbettung und Interaktion ermöglicht.

⟨RM5⟩ Ein gemeinsamer Kalibrierungsprozess muss gewährleistet sein, um eine einheitliche Ausrichtung aller Inhalte sicherzustellen.

Diese Funktion wird durch die Komponenten <C60> **Netzwerkmanager** zur Geräteerkennung und Verbindungsinitialisierung sowie <C20> **Client Logic** zur Synchronisierung der Kalibrierungsdaten umgesetzt.

9.2 Sollkriterien

⟨RS1⟩ Das System sollte Cloud-Dienste zur Datenspeicherung und für geräte- sowie sitzungs-übergreifenden Zugriff nutzen.

Diese Anforderung wird **nicht umgesetzt**, da das System ausschließlich auf lokale Netzwerkverbindungen und einen zentralen HTTP-Server setzt.

⟨RS2⟩ Die Plattform sollte eine große Auswahl an Filmen oder multimedialen Inhalten bereitstellen.

Dies wird durch die Kombination der Komponenten <C70> **Video Server** zur Bereitstellung der Inhalte, <C50> **Video Player** zur lokalen Wiedergabe und <C40> **UI Interface** zur Anzeige und Auswahl realisiert.

⟨RS3⟩ Nutzer sollten in der Lage sein, in der VR-/MR-Umgebung Inhalte zu zoomen und zu erkunden.

Diese Interaktionsmöglichkeit wird über die <C40> **UI Interface** sowie die <C20> **Client Logic** unterstützt, wobei insbesondere Benutzerinteraktionen mit der Szene ermöglicht werden.

⟨RS4⟩ Es sollte möglich sein, Avatare oder Charaktere individuell zu gestalten.

Diese Funktion ist derzeit **nicht umgesetzt**, kann jedoch durch Erweiterung der UI-Komponente in künftigen Versionen integriert werden.

⟨RS5⟩ Das System sollte anpassbare Umgebungen ermöglichen, um Immersion und Selbstausdruck zu fördern.

Diese Funktion wird durch die Komponenten <C40> **UI Interface** (für die Auswahl) und <C50> **Video Player** umgesetzt.

9.3 Kannkriterien

⟨RC1⟩ Die Plattform kann ein Emote-Rad enthalten, um während der Sitzung vorgefertigte Gesten oder Reaktionen auszulösen.

Diese optionale Funktion wird über die Komponenten <C40> **UI Interface** zur Anzeige und Eingabe sowie <C20> **Client Logic** zur Verarbeitung umgesetzt.

⟨RC2⟩ Das System kann auswählbare Gesichtsausdrücke unterstützen, um die Ausdruckskraft der Avatare zu erhöhen.

Diese Funktion ist **geplant**, wurde im aktuellen Entwicklungsstand jedoch noch **nicht implementiert**.

⟨RC3⟩ Ein Speicherpunktsystem kann integriert werden, um Sitzungsfortschritt zu speichern und später fortzusetzen.

Diese Funktion ist derzeit **nicht umgesetzt** und wurde aufgrund der Projektprioritäten zurückgestellt.

⟨RC4⟩ Eine Nachbesprechungsfunktion kann zur Verfügung stehen, um z.B. am Ende einer Sitzung die Umgebung stimmungsvoll zu verändern.

Auch diese Funktion wurde **nicht umgesetzt**, stellt aber eine sinnvolle Erweiterungsmöglichkeit dar.

⟨RC5⟩ Die Anwendung kann Hand-Tracking unterstützen, um die Steuerung ohne Controller zu ermöglichen.

Diese Möglichkeit wird durch die <C40> **UI Interface** realisiert, die Eingaben sowohl per Hand-Tracking als auch via Controller akzeptiert.

⟨RC6⟩ Das System kann eine ortsunabhängige Multiplayer-Architektur bieten.

Im aktuellen Stand wird dies **teilweise** durch die <C60> **Netzwerkmanager** und den <C70> **Video Server** realisiert, allerdings ist der Betrieb momentan auf dasselbe lokale WLAN beschränkt.

10 Glossar

OpenXR-Plugin: Eine Schnittstelle, die es der Godot Engine ermöglicht, auf XR-Funktionen (Virtual und Mixed Reality) verschiedener Geräte zuzugreifen. Sie sorgt dafür, dass VR/MR-Anwendungen plattformübergreifend laufen – z.B. auf der Meta Quest 3.

Godot-Szenendateien (.tscn): Textbasierte Dateien der Godot Engine, die die Struktur und Eigenschaften von Szenen enthalten – z.B. UI-Elemente, 3D-Objekte oder Skripte. Sie bilden die Grundlage jeder Ansicht oder Interaktion in der Anwendung.

UI-Zustände: Verschiedene Anzeige- und Interaktionsmodi der Benutzeroberfläche (UI), z.B. Hauptmenü, Filmauswahl oder Startbildschirm. Sie steuern, welche Elemente sichtbar sind und wie Nutzer mit ihnen interagieren können.

.apk-Datei: Installationsdatei für Android-Anwendungen. Sie enthält alle nötigen Daten, um eine App auf Geräten wie der Meta Quest 3 zu installieren.

SideQuest: Ein Tool, mit dem .apk-Dateien bequem von einem PC auf die Meta Quest übertragen und dort installiert werden können – nützlich zum Exportieren und Testen eigener Anwendungen.

HTTP-Server: Ein HTTP-Server ist ein Programm, das Dateien (z.B. Webseiten oder Videos) über das Netzwerk bereitstellt. Geräte können über eine IP-Adresse und einen Webbrowser oder eine App auf die Inhalte zugreifen.

UDP (User Datagram Protocol) Ein schnelles, verbindungsloses Netzwerkprotokoll, das Daten ohne Zustellgarantie versendet. In diesem Projekt wird es für die Host-Erkennung im lokalen Netzwerk verwendet.

ENet Eine schlanke Netzbibliothek, die auf UDP basiert. Sie bietet zusätzliche Funktionen wie Paket-Reihenfolge und Zuverlässigkeit und wird hier für die Multiplayer-Kommunikation und Synchronisation über RPCs genutzt.

RPC (Remote Procedure Call) Ein Mechanismus, mit dem ein Gerät eine Funktion auf einem anderen Gerät über das Netzwerk ausführen kann. Zum Beispiel kann der Host mit einem „Play“-Klick bewirken, dass alle Clients gleichzeitig den Film starten.